Rareminds Admin App - Complete Documentation

📋 Table of Contents

---

1. Admin App Overview

Purpose

The Rareminds Admin App is a **Next.js-based administrative interface** exclusively for platform administrators (RM Admin) to manage the entire Rareminds ecosystem. This app provides complete oversight and control over all entities, users, and platform operations.

Key Characteristics

- **Exclusive Access**: Only accessible to RM Admin users (platform_admin role)
- **Separate Deployment**: Independent from the main platform app
- **Dedicated Backend**: Uses Backend API 1 (Admin API)
- **Full Platform Control**: Manages all five entity types and their hierarchies

What Admin App Does

1. **Entity Management**: Create, approve, monitor, and manage Schools, Colleges, Universities, and Companies
2. **User Oversight**: View and manage all platform users across all entities
3. **Platform Analytics**: Dashboard with comprehensive platform statistics
4. **Approval Workflows**: Approve/reject entity registrations
5. **Audit & Monitoring**: Track all platform activities and changes

6. **System Configuration**: Manage platform-wide settings and permissions

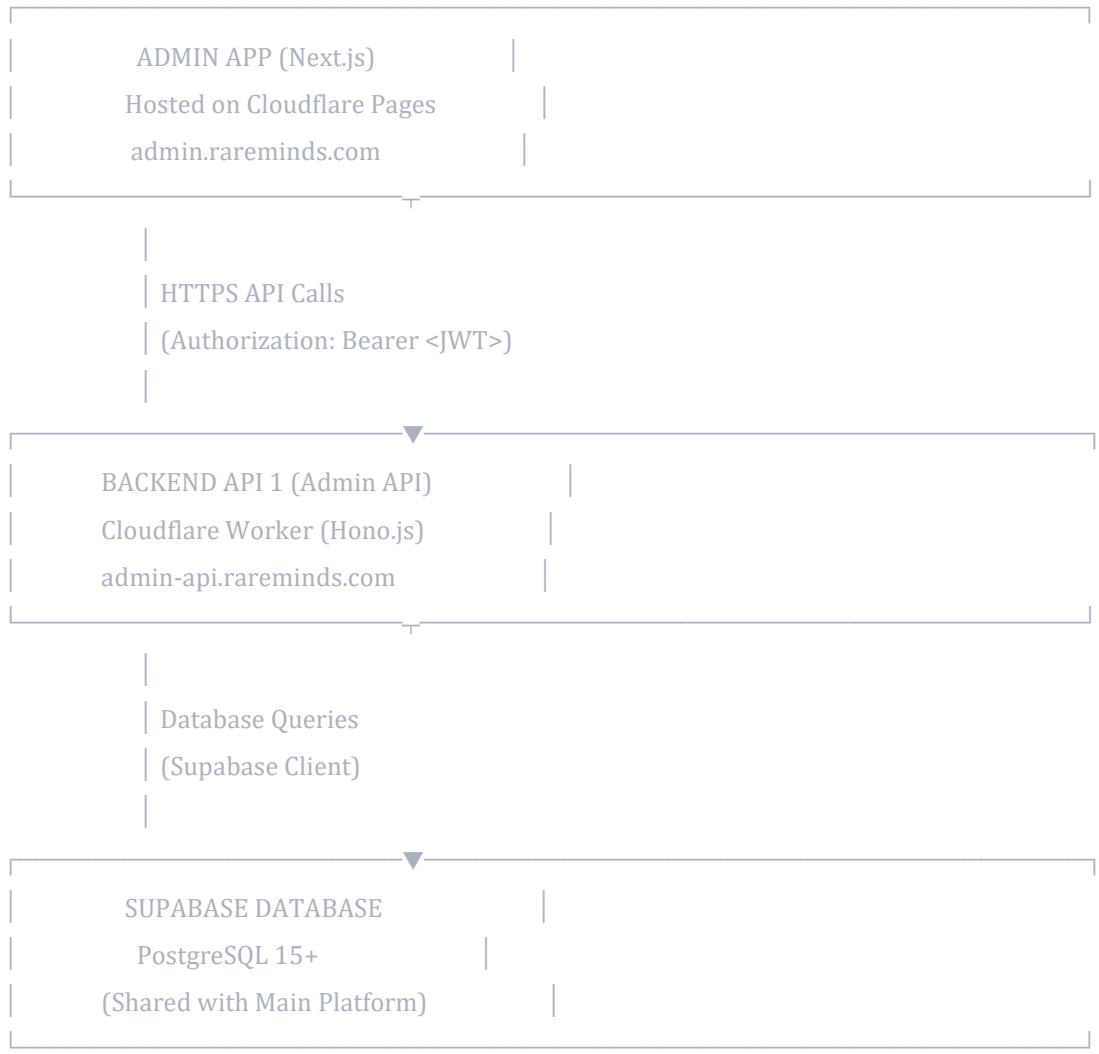
What Admin App Does NOT Do

- Does NOT handle day-to-day operations of Schools/Colleges/Universities/Companies
- Does NOT create students, educators, or recruiters (that's done in the Main Platform App)
- Does NOT handle learning materials or recruitment activities
- Does NOT provide student/educator/recruiter interfaces

---

2. Admin App Architecture

High-Level Architecture




```

### Admin App Components
```

```
Admin App
│
├── Authentication Layer
│   ├── Login Page
│   ├── JWT Token Management
│   └── Session Handling
│
├── Dashboard
│   ├── Platform Statistics
│   ├── Recent Activities
│   ├── Pending Approvals
│   └── Quick Actions
│
├── Entity Management
│   ├── Schools Module
│   ├── Colleges Module
│   ├── Universities Module
│   └── Companies Module
│
├── User Management
│   ├── All Users List
│   ├── User Details
│   └── User Actions (Suspend/Activate)
│
├── Approval Center
│   ├── Pending Schools
│   ├── Pending Colleges
│   ├── Pending Universities
│   └── Pending Companies
│
└── Audit & Reports
    ├── Audit Logs
    ├── Activity Reports
    └── Platform Analytics
```

---

3. Technology Stack

Frontend (Admin App)

typescript

```
{
  "framework": "Next.js 14+",
  "appRouter": true,
  "language": "TypeScript 5+",
  "styling": "Tailwind CSS 3.4+",
  "uiLibrary": "shadcn/ui",
  "stateManagement": "Zustand",
  "formHandling": "React Hook Form + Zod",
  "dataFetching": "TanStack Query (React Query)",
  "charts": "Recharts",
  "tables": "TanStack Table",
  "notifications": "React Hot Toast",
  "icons": "Lucide React"
}
```

Backend (Admin API)

typescript

```
{
  "platform": "Cloudflare Workers",
  "framework": "Hono.js",
  "language": "TypeScript",
  "database": "@supabase/supabase-js",
  "auth": "jose (JWT)",
  "validation": "Zod",
  "cors": "hono/cors"
}
```

Development Tools

typescript

```
{
  "packageManager": "pnpm",
  "codeQuality": ["ESLint", "Prettier", "TypeScript"],
  "testing": ["Vitest", "Playwright"],
  "versionControl": "Git + GitHub",
  "ci/cd": "GitHub Actions"
}
```

```

### Hosting & Infrastructure
```
- Frontend: Cloudflare Pages
- Backend API: Cloudflare Workers
- Database: Supabase (PostgreSQL)
- File Storage: Cloudflare R2 or Supabase Storage
- CDN: Cloudflare CDN (automatic)
- SSL: Automatic (Cloudflare)

---

4. Admin User Roles & Permissions

Role: platform_admin

Description: The only role with access to the Admin App. Has complete control over the entire platform.

Permissions:

typescript
```
const PLATFORM_ADMIN_PERMISSIONS = [
 // Platform-wide
 'platform:manage_all',
 'platform:view_analytics',
 'platform:configure_settings',

 // Schools
 'school:create',
 'school:read',
 'school:update',
 'school:delete',
 'school:approve',
 'school:reject',
 'school:suspend',

 // Colleges (Standalone)
 'college:create',
 'college:read',
 'college:update',
```

```
  'college:delete',
  'college:approve',
  'college:reject',
  'college:suspend',

  // Universities
  'university:create',
  'university:read',
  'university:update',
  'university:delete',
  'university:approve',
  'university:reject',
  'university:suspend',

  // Companies
  'company:create',
  'company:read',
  'company:update',
  'company:delete',
  'company:approve',
  'company:reject',
  'company:suspend',

  // Users (all types)
  'user:read_all',
  'user:update_any',
  'user:delete_any',
  'user:suspend_any',
  'user:activate_any',

  // Audit & Logs
  'audit:read_all',
  'logs:read_all',

  // Permissions Management
  'permission:manage',
  'role:manage'
];
```

Authentication Requirements

typescript

```typescript
// Admin users must have:
interface AdminUser {
  role: 'platform_admin';
  entity_type: null;       // Admin is not tied to any entity
  entity_id: null;
  account_status: 'active';
  permissions: string[];    // All platform_admin permissions
}
```

---

5. Admin App Features

5.1 Dashboard

Purpose: Overview of entire platform at a glance

Components:

- **Statistics Cards**:
  - Total Schools (Active, Pending, Suspended)
  - Total Colleges (Active, Pending, Suspended)
  - Total Universities (Active, Pending, Suspended)
  - Total Companies (Active, Pending, Suspended)
  - Total Users by Role
  - Total Students across all entities
  - Total Educators/Lecturers
  - Total Recruiters
- **Charts**:
  - User Growth Over Time (line chart)
  - Entity Distribution (pie chart)
  - Entity Status Breakdown (bar chart)
  - Monthly Registration Trends
- **Recent Activities**:
  - Latest entity registrations
  - Recent approvals/rejections
  - Recent user creations
  - System alerts
- **Pending Approvals**:
  - Quick counts of pending entities
  - Direct links to approval workflows

5.2 School Management

Features:

- List all schools with filters (status, location, date)
- View detailed school information
- Approve/Reject school registrations
- Suspend/Activate schools
- Edit school details
- View school's classes, educators, and students
- Delete schools (with cascading considerations)

School Details View:

typescript

```typescript
interface SchoolDetailsView {
 basicInfo: {
  name: string;
  code: string;
  address: string;
  contactInfo: ContactDetails;
  establishedYear: number;
  board: string;
 };
 status: {
  accountStatus: 'active' | 'inactive' | 'suspended' | 'pending';
  approvalStatus: 'approved' | 'rejected' | 'pending';
  approvedBy?: string;
  approvedAt?: Date;
 };
 stats: {
  totalClasses: number;
  totalEducators: number;
  totalStudents: number;
 };
 adminUser: {
  name: string;
  email: string;
  lastLogin?: Date;
 };
 auditTrail: AuditLog[];
}
```

5.3 College Management

Features:

- List all standalone colleges
- View college details
- Approve/Reject college registrations
- Suspend/Activate colleges
- Edit college information
- View college's courses, lecturers, and students
- Delete colleges

College Details View:

typescript

```typescript
interface CollegeDetailsView {
 basicInfo: {
  name: string;
  code: string;
  address: string;
  contactInfo: ContactDetails;
  affiliation: string;
  accreditation: string;
 };
 status: {
  accountStatus: AccountStatus;
  approvalStatus: ApprovalStatus;
 };
 stats: {
  totalCourses: number;
  totalLecturers: number;
  totalStudents: number;
 };
 adminUser: AdminUserInfo;
 auditTrail: AuditLog[];
}
```

5.4 University Management

Features:

- List all universities
- View university details
- Approve/Reject university registrations
- Suspend/Activate universities
- Edit university information
- View university's colleges (departments)
- View aggregated stats (total courses, lecturers, students across all colleges)

- Delete universities

University Details View:

typescript

```typescript
interface UniversityDetailsView {
  basicInfo: {
    name: string;
    code: string;
    address: string;
    contactInfo: ContactDetails;
    universityType: string;
    accreditation: string;
  };
  status: {
    accountStatus: AccountStatus;
    approvalStatus: ApprovalStatus;
  };
  hierarchy: {
    totalColleges: number;
    colleges: CollegeSummary[];
  };
  aggregateStats: {
    totalCourses: number;
    totalLecturers: number;
    totalStudents: number;
  };
  adminUser: AdminUserInfo;
  auditTrail: AuditLog[];
}
```

5.5 Company Management

Features:

- List all companies
- View company details
- Approve/Reject company registrations
- Suspend/Activate companies
- Edit company information
- View company's branches
- View all recruiters (HQ and branch-level)
- Delete companies

Company Details View:

typescript

```
interface CompanyDetailsView {
 basicInfo: {
  name: string;
  code: string;
  industry: string;
  companySize: string;
  headquarters: AddressInfo;
  contactPerson: ContactPersonInfo;
 };
 status: {
  accountStatus: AccountStatus;
  approvalStatus: ApprovalStatus;
 };
 structure: {
  totalBranches: number;
  branches: BranchSummary[];
 };
 stats: {
  totalRecruiters: number;
  hqRecruiters: number;
  branchRecruiters: number;
 };
 adminUser: AdminUserInfo;
 auditTrail: AuditLog[];
}
```

5.6 User Management

Features:

- View all users across all entities
- Filter by role, entity type, status
- Search by name, email
- View user details
- Suspend/Activate user accounts
- Delete user accounts
- Reset user passwords
- View user's activity logs

User List Filters:

typescript

```typescript
interface UserFilters {
  role?: UserRole;
  entityType?: EntityType;
  accountStatus?: AccountStatus;
  searchQuery?: string;
  dateRange?: { from: Date; to: Date };
  sortBy?: 'created_at' | 'last_login' | 'email';
  sortOrder?: 'asc' | 'desc';
}
```

5.7 Approval Center

Purpose: Centralized view of all pending entity registrations

Sections:

1. **Pending Schools**
   - List with basic info
   - Quick approve/reject actions
   - Bulk actions
2. **Pending Colleges**
   - Same as schools
3. **Pending Universities**
   - Same as schools
4. **Pending Companies**
   - Same as schools

Approval Workflow:

typescript

```typescript
interface ApprovalAction {
  entityId: string;
  entityType: 'school' | 'college_standalone' | 'university' | 'company';
  action: 'approve' | 'reject';
  reason?: string;  // Required for rejection
  notes?: string;
}
```

5.8 Audit Logs

Features:

- View all platform activities
- Filter by user, action type, resource, date

- Export audit logs
- Search logs

Audit Log Entry:

typescript

```typescript
interface AuditLogEntry {
  id: string;
  userId: string;
  userName: string;
  action: string;
  resourceType: string;
  resourceId: string;
  oldValues?: Record<string, any>;
  newValues?: Record<string, any>;
  ipAddress: string;
  userAgent: string;
  createdAt: Date;
}
```

5.9 Platform Analytics

Reports Available:

- User growth trends
- Entity distribution by type and location
- Active vs inactive entities
- Student enrollment statistics
- Educator/Lecturer distribution
- Company recruitment activity
- Platform usage metrics

---

6. Database Schema for Admin Operations

Key Tables Used by Admin App

sql

```sql
-- Users table (admin reads all)
SELECT * FROM users WHERE role = 'platform_admin';


-- Schools management
SELECT * FROM schools;
```

```
SELECT * FROM school_classes WHERE school_id = ?;
SELECT * FROM school_educators WHERE school_id = ?;

-- Colleges management
SELECT * FROM colleges_standalone;
SELECT * FROM college_courses WHERE college_id = ?;
SELECT * FROM college_lecturers WHERE college_id = ?;

-- Universities management
SELECT * FROM universities;
SELECT * FROM university_colleges WHERE university_id = ?;
SELECT * FROM university_courses WHERE college_id = ?;
SELECT * FROM university_lecturers WHERE college_id = ?;

-- Companies management
SELECT * FROM companies;
SELECT * FROM company_branches WHERE company_id = ?;
SELECT * FROM recruiters WHERE company_id = ?;

-- Students (across all entities)
SELECT * FROM students;

-- Audit logs
SELECT * FROM audit_logs;
```

Common Queries

1. Get Dashboard Statistics:

sql

```
-- Total entities by type and status
SELECT
  'school' as entity_type,
  account_status,
  COUNT(*) as count
FROM schools
GROUP BY account_status

UNION ALL
```

```sql
SELECT
  'college' as entity_type,
  account_status,
  COUNT(*) as count
FROM colleges_standalone
GROUP BY account_status

-- Similar for universities and companies
```

2. Get Pending Approvals:

sql

```sql
SELECT
  id,
  name,
  code,
  email,
  created_at,
  'school' as entity_type
FROM schools
WHERE approval_status = 'pending'

UNION ALL

SELECT
  id,
  name,
  code,
  email,
  created_at,
  'college' as entity_type
FROM colleges_standalone
WHERE approval_status = 'pending'

-- Similar for universities and companies
```

3. Get User Distribution:

sql

```sql
SELECT
  role,
```

```
  account_status,
  COUNT(*) as count
FROM users
GROUP BY role, account_status
ORDER BY role;
```

---

7. Admin API Endpoints

Base URL
```
Production: https://admin-api.rareminds.com
Staging: https://admin-api-staging.rareminds.com
Development: http://localhost:8787
```

Authentication Endpoints

typescript
```
POST   /api/auth/login
POST   /api/auth/logout
POST   /api/auth/refresh
GET    /api/auth/me
```

Dashboard Endpoints

typescript
```
GET    /api/admin/dashboard
// Response: Platform-wide statistics

GET    /api/admin/stats
// Response: Detailed analytics data
```

School Management

typescript
```
GET    /api/admin/schools
// Query params: status, city, state, page, limit, search
// Response: Paginated list of schools

POST   /api/admin/schools
```

// Body: School registration data

// Response: Created school

GET /api/admin/schools/:id

// Response: School details with related data

PUT /api/admin/schools/:id

// Body: Updated school data

// Response: Updated school

DELETE /api/admin/schools/:id

// Response: Success message

POST /api/admin/schools/:id/approve

// Body: { notes?: string }

// Response: Approved school

POST /api/admin/schools/:id/reject

// Body: { reason: string }

// Response: Rejected school

POST /api/admin/schools/:id/suspend

// Body: { reason: string }

// Response: Suspended school

POST /api/admin/schools/:id/activate

// Response: Activated school

GET /api/admin/schools/:id/classes

// Response: School's classes

GET /api/admin/schools/:id/educators

// Response: School's educators

GET /api/admin/schools/:id/students

// Response: School's students

GET /api/admin/schools/:id/audit-logs

// Response: School's audit trail

College Management

typescript

```
GET    /api/admin/colleges
POST   /api/admin/colleges
GET    /api/admin/colleges/:id
PUT    /api/admin/colleges/:id
DELETE /api/admin/colleges/:id
POST   /api/admin/colleges/:id/approve
POST   /api/admin/colleges/:id/reject
POST   /api/admin/colleges/:id/suspend
POST   /api/admin/colleges/:id/activate
GET    /api/admin/colleges/:id/courses
GET    /api/admin/colleges/:id/lecturers
GET    /api/admin/colleges/:id/students
GET    /api/admin/colleges/:id/audit-logs
```

University Management

typescript

```
GET    /api/admin/universities
POST   /api/admin/universities
GET    /api/admin/universities/:id
PUT    /api/admin/universities/:id
DELETE /api/admin/universities/:id
POST   /api/admin/universities/:id/approve
POST   /api/admin/universities/:id/reject
POST   /api/admin/universities/:id/suspend
POST   /api/admin/universities/:id/activate
GET    /api/admin/universities/:id/colleges
GET    /api/admin/universities/:id/stats
GET    /api/admin/universities/:id/audit-logs
```

Company Management

typescript

```
GET    /api/admin/companies
POST   /api/admin/companies
GET    /api/admin/companies/:id
PUT    /api/admin/companies/:id
DELETE /api/admin/companies/:id
```

```
POST   /api/admin/companies/:id/approve
POST   /api/admin/companies/:id/reject
POST   /api/admin/companies/:id/suspend
POST   /api/admin/companies/:id/activate
GET    /api/admin/companies/:id/branches
GET    /api/admin/companies/:id/recruiters
GET    /api/admin/companies/:id/audit-logs
```

User Management

typescript

```
GET    /api/admin/users
// Query params: role, entity_type, status, search, page, limit
// Response: Paginated user list

GET    /api/admin/users/:id
// Response: User details

PUT    /api/admin/users/:id
// Body: Updated user data
// Response: Updated user

DELETE /api/admin/users/:id
// Response: Success message

POST   /api/admin/users/:id/suspend
// Body: { reason: string }
// Response: Suspended user

POST   /api/admin/users/:id/activate
// Response: Activated user

POST   /api/admin/users/:id/reset-password
// Response: Password reset link sent

GET    /api/admin/users/:id/audit-logs
// Response: User's activity logs
```

Approval Center

typescript

GET /api/admin/approvals/pending

// Response: All pending approvals grouped by entity type

GET /api/admin/approvals/schools

// Response: Pending schools

GET /api/admin/approvals/colleges

// Response: Pending colleges

GET /api/admin/approvals/universities

// Response: Pending universities

GET /api/admin/approvals/companies

// Response: Pending companies

POST /api/admin/approvals/bulk-action

// Body: { entityIds: string[], action: 'approve' | 'reject', reason?: string }

// Response: Bulk action results

Audit Logs

typescript

GET /api/admin/audit-logs

// Query params: userId, action, resourceType, dateFrom, dateTo, page, limit

// Response: Paginated audit logs

GET /api/admin/audit-logs/:id

// Response: Audit log details

GET /api/admin/audit-logs/export

// Query params: Same as GET /audit-logs

// Response: CSV file download

Analytics

typescript

GET /api/admin/analytics/users

// Response: User growth data

GET /api/admin/analytics/entities

// Response: Entity distribution data

```
GET    /api/admin/analytics/activity
// Response: Platform activity metrics

GET    /api/admin/analytics/reports
// Query params: reportType, dateFrom, dateTo
// Response: Custom report data
```

---

8. Admin App Pages & Components

Page Structure

```
/app
├── (auth)
│   └── login
│       └── page.tsx              // Login page
│
├── (dashboard)
│   ├── layout.tsx                // Dashboard layout with sidebar
│   ├── page.tsx                  // Dashboard home
│   │
│   ├── schools
│   │   ├── page.tsx              // Schools list
│   │   ├── [id]
│   │   │   ├── page.tsx          // School details
│   │   │   ├── classes
│   │   │   │   └── page.tsx      // School classes
│   │   │   ├── educators
│   │   │   │   └── page.tsx      // School educators
│   │   │   └── students
│   │   │       └── page.tsx      // School students
│   │   └── create
│   │       └── page.tsx          // Create school
│   │
│   ├── colleges
│   │   ├── page.tsx              // Colleges list
```

```
│   │   ├── [id]
│   │   │   ├── page.tsx            // College details
│   │   │   ├── courses
│   │   │   │   └── page.tsx        // College courses
│   │   │   ├── lecturers
│   │   │   │   └── page.tsx        // College lecturers
│   │   │   └── students
│   │   │       └── page.tsx        // College students
│   │   └── create
│   │       └── page.tsx            // Create college
│   │
│   ├── universities
│   │   ├── page.tsx                // Universities list
│   │   ├── [id]
│   │   │   ├── page.tsx            // University details
│   │   │   ├── colleges
│   │   │   │   ├── page.tsx        // University colleges
│   │   │   │   └── [collegeId]
│   │   │   │       ├── page.tsx    // College within university
│   │   │   │       └── courses
│   │   │   │           └── page.tsx // Courses in college
│   │   │   └── stats
│   │   │       └── page.tsx        // University statistics
│   │   └── create
│   │       └── page.tsx            // Create university
│   │
│   ├── companies
│   │   ├── page.tsx                // Companies list
│   │   ├── [id]
│   │   │   ├── page.tsx            // Company details
│   │   │   ├── branches
│   │   │   │   └── page.tsx        // Company branches
│   │   │   └── recruiters
│   │   │       └── page.tsx        // Company recruiters
│   │   └── create
│   │       └── page.tsx            // Create company
│   │
│   ├── users
│   │   ├── page.tsx                // All users list
```

```
│   │   └── [id]
│   │       └── page.tsx          // User details
│   │
│   ├── approvals
│   │   ├── page.tsx              // Approval center
│   │   ├── schools
│   │   │   └── page.tsx          // Pending schools
│   │   ├── colleges
│   │   │   └── page.tsx          // Pending colleges
│   │   ├── universities
│   │   │   └── page.tsx          // Pending universities
│   │   └── companies
│   │       └── page.tsx          // Pending companies
│   │
│   ├── audit-logs
│   │   ├── page.tsx              // Audit logs list
│   │   └── [id]
│   │       └── page.tsx          // Audit log details
│   │
│   └── analytics
│       ├── page.tsx              // Analytics dashboard
│       ├── users
│       │   └── page.tsx          // User analytics
│       ├── entities
│       │   └── page.tsx          // Entity analytics
│       └── reports
│           └── page.tsx          // Custom reports
│
└── api
    └── (tRPC or direct API routes if needed)
```

Key Components

Before I continue with the components section, I have some questions:

1. **UI Component Library**: Should I use shadcn/ui for all components, or do you have a preference for another library (e.g., Ant Design, Material-UI)?
2. **Data Tables**: For the entity lists (schools, colleges, etc.), would you like:
    - Server-side pagination and sorting?
    - Client-side filtering with server-side data?
    - Infinite scroll or traditional pagination?
3. **Form Handling**: For creating/editing entities:

    - Should forms be multi-step wizards or single-page forms?
    - Do you want inline validation or validation on submit?
4. **Confirmation Dialogs**: For critical actions (delete, suspend, reject):
    - Should these require typing a confirmation phrase?
    - Or just a simple "Are you sure?" dialog?
5. **Real-time Updates**: Should the admin dashboard show real-time updates (e.g., new pending approvals) using WebSockets or polling?

Please let me know your preferences, and I'll continue with the detailed component documentation!

2 / 2
Retry
D

1. use shadcn/ui
2. server side sorting with Infinite scroll
3. single-page forms, inline validation
4. just a simple "Are you sure?"
5. yes

don't include component code in the doc

Rareminds Admin App - Complete Documentation (Continued)

8. Admin App Pages & Components (Continued)

Key Components

8.1 Layout Components

DashboardLayout

- Wrapper for all dashboard pages
- Includes sidebar navigation
- Top header with user profile and notifications
- Real-time notification system for pending approvals
- Breadcrumb navigation
- Responsive design (mobile-friendly sidebar)

Sidebar

- Navigation menu with icons
- Active route highlighting
- Collapsible sections
- Quick stats badges (e.g., pending approvals count)

- Logout button

Header

- User profile dropdown
- Notification bell with real-time updates
- Global search (searches across all entities)
- Theme toggle (dark/light mode)

8.2 Dashboard Components

StatsCard

- Displays single metric with icon
- Shows comparison with previous period
- Color-coded for different statuses
- Click to drill down

EntityDistributionChart

- Pie chart showing entity types distribution
- Interactive legends
- Tooltips with detailed numbers

UserGrowthChart

- Line chart showing user growth over time
- Multiple series (by role)
- Date range selector

RecentActivityFeed

- Real-time activity updates
- Grouped by time (Today, Yesterday, etc.)
- User avatars and action icons
- Click to view details

PendingApprovalsList

- Quick view of pending entities
- Shows entity type, name, date submitted
- Quick approve/reject buttons
- Link to full approval workflow

8.3 Entity Management Components

EntityTable

- Reusable table for all entity types
- Server-side sorting

- Infinite scroll pagination
- Multi-select for bulk actions
- Column filters
- Export to CSV functionality
- Quick action buttons (view, edit, delete)

EntityDetailsCard

- Shows comprehensive entity information
- Tabbed interface for different sections
- Status badges
- Action buttons (approve, reject, suspend, etc.)
- Edit mode toggle

EntityStatusBadge

- Color-coded status indicator
- Shows both account_status and approval_status
- Tooltip with status change history

EntityForm

- Single-page form with sections
- Inline validation using Zod
- Auto-save draft functionality
- File upload for documents
- Address autocomplete
- Conditional fields based on entity type

8.4 Approval Components

ApprovalQueue

- Lists pending entities
- Filters by entity type and date
- Bulk selection and actions
- Preview panel for quick review

ApprovalDialog

- Modal for approve/reject actions
- Text area for notes/reason
- Confirmation with simple "Are you sure?"
- Shows entity summary

ApprovalHistory

- Timeline of approval actions
- Shows who approved/rejected and when
- Displays reasons for rejection

8.5 User Management Components

UserTable

- Similar to EntityTable
- Filters by role, status, entity type
- Quick actions (suspend, activate, reset password)
- User avatar and online status

UserDetailsPanel

- Shows user information
- Entity association details
- Activity history
- Account actions

UserActivityTimeline

- Chronological list of user actions
- Grouped by date
- Icons for different action types

8.6 Audit Log Components

AuditLogTable

- Searchable and filterable table
- Shows action, user, resource, timestamp
- Expandable rows for old/new values comparison
- Export functionality

AuditLogFilter

- Date range picker
- User selector
- Action type dropdown
- Resource type dropdown

AuditLogDetails

- Side panel or modal
- JSON diff viewer for changes
- IP address and user agent information
- Related audit entries

8.7 Analytics Components

AnalyticsDashboard

- Multiple chart types

- Date range selector
- Export reports functionality
- Custom report builder

MetricCard

- Single KPI display
- Trend indicator (up/down)
- Sparkline for quick trend view
- Comparison period selector

ReportGenerator

- Form to configure custom reports
- Metric selection
- Date range selection
- Export format options (CSV, PDF)

8.8 Common/Shared Components

ConfirmDialog

- Simple confirmation modal
- "Are you sure?" message
- Customizable title and description
- Action buttons (Confirm/Cancel)

DataTable

- Generic reusable table component
- Built with TanStack Table
- Server-side sorting and filtering
- Infinite scroll
- Column visibility toggle
- Column resizing

SearchBar

- Global search input
- Debounced search
- Autocomplete suggestions
- Recent searches

StatusFilter

- Dropdown for status filtering
- Multi-select
- Shows count per status

DateRangePicker

- Custom date range selection
- Preset ranges (Today, Last 7 days, Last 30 days, etc.)
- Custom range input

FileUploader

- Drag and drop file upload
- Progress indicator
- File type validation
- Preview for images

LoadingState

- Skeleton loaders for different content types
- Spinner for async actions
- Progress bar for multi-step processes

EmptyState

- Shown when no data available
- Contextual message and illustration
- Call-to-action button

ErrorBoundary

- Catches and displays errors gracefully
- "Something went wrong" message
- Retry button
- Error details (in development)

8.9 Form Components

FormField

- Wrapper for form inputs with label and error
- Consistent styling
- Required indicator

FormSection

- Groups related form fields
- Collapsible sections
- Section headings

AutocompleteInput

- Input with autocomplete suggestions
- Async data fetching
- Keyboard navigation

MultiSelect

- Select multiple options
- Tag display for selected items
- Search within options

---

9. Authentication & Authorization Flow

9.1 Login Flow

```
1. Admin visits admin.rareminds.com
   ↓
2. Redirected to /login (if not authenticated)
   ↓
3. Admin enters email and password
   ↓
4. Frontend sends POST /api/auth/login to Admin API
   ↓
5. Admin API validates credentials with Supabase Auth
   ↓
6. Admin API checks user role = 'platform_admin'
   ↓
7. If valid, Admin API generates JWT token with permissions
   ↓
8. Admin API returns JWT token
   ↓
9. Frontend stores token in httpOnly cookie
   ↓
10. Redirect to dashboard
```

9.2 Token Structure

typescript

```typescript
interface AdminJWTPayload {
  sub: string;                // user_id
  email: string;
  role: 'platform_admin';
  entity_type: null;          // Admin has no entity
  entity_id: null;
  permissions: string[];      // All platform_admin permissions
```

```
  iat: number;                // Issued at
  exp: number;                // Expires at (24 hours)
  iss: 'rareminds-admin-api';
  aud: 'rareminds-admin-app';
}
```

9.3 Protected Routes

typescript

```
// middleware.ts
export function middleware(request: NextRequest) {
  const token = request.cookies.get('admin_token');

  // Public routes (don't need authentication)
  const publicRoutes = ['/login'];

  if (publicRoutes.includes(request.nextUrl.pathname)) {
    return NextResponse.next();
  }

  // No token, redirect to login
  if (!token) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // Verify token validity (check expiry, signature)
  try {
    const decoded = verifyToken(token.value);

    // Check if user is platform_admin
    if (decoded.role !== 'platform_admin') {
      return NextResponse.redirect(new URL('/unauthorized', request.url));
    }

    return NextResponse.next();
  } catch (error) {
    // Invalid token, clear cookie and redirect
    const response = NextResponse.redirect(new URL('/login', request.url));
    response.cookies.delete('admin_token');
    return response;
```

```
  }
}

export const config = {
  matcher: ['/((?!api|_next/static|_next/image|favicon.ico).*)']
};
```

9.4 API Authentication

typescript

```
// Every API call includes token
const apiClient = {
  async request(endpoint: string, options: RequestInit = {}) {
    const token = getCookie('admin_token');

    const response = await fetch(`${API_BASE_URL}${endpoint}`, {
      ...options,
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`,
        ...options.headers
      },
      credentials: 'include'
    });

    // Handle 401 - token expired
    if (response.status === 401) {
      // Try to refresh token
      const refreshed = await refreshToken();
      if (refreshed) {
        // Retry original request with new token
        return apiClient.request(endpoint, options);
      } else {
        // Refresh failed, redirect to login
        window.location.href = '/login';
        throw new Error('Session expired');
      }
    }

    if (!response.ok) {
```

```
    const error = await response.json();
    throw new Error(error.error.message);
  }

  return response.json();
 }
};
```

9.5 Token Refresh Flow

```
1. Access token expires (after 24 hours)
  ↓
2. API returns 401 Unauthorized
  ↓
3. Frontend attempts token refresh
  ↓
4. POST /api/auth/refresh with refresh token (from httpOnly cookie)
  ↓
5. Admin API validates refresh token
  ↓
6. Admin API generates new access token
  ↓
7. Frontend receives new token
  ↓
8. Retry original request with new token
  ↓
9. If refresh fails, redirect to login
```

9.6 Logout Flow

```
1. Admin clicks logout button
  ↓
2. Frontend sends POST /api/auth/logout
  ↓
3. Admin API invalidates refresh token (blacklist in KV store)
  ↓
4. Frontend clears cookies (access and refresh tokens)
```

```
  ↓
5. Redirect to /login
```

9.7 Session Management

typescript

```
// Session timeout after 24 hours of inactivity
interface Session {
  userId: string;
  email: string;
  role: 'platform_admin';
  lastActivity: Date;
  expiresAt: Date;
}

// Activity tracking
function trackActivity() {
  // Update lastActivity on every API call
  // If inactive for > 24 hours, force logout
}

// Idle timeout warning
function showIdleWarning() {
  // Show modal: "You will be logged out in 5 minutes due to inactivity"
  // Give option to "Stay Logged In"
}
```

---

10. Admin Workflows

10.1 Entity Registration & Approval Workflow

School Registration Workflow

```
Step 1: Entity Submits Registration
 ├── School admin fills registration form on main platform
 ├── Data includes: name, code, address, contact info, documents
 ├── Status: approval_status = 'pending', account_status = 'pending'
```

```
└── Notification sent to RM Admin

Step 2: RM Admin Reviews
├── RM Admin sees new entry in "Pending Approvals"
├── Clicks to view school details
├── Reviews submitted information and documents
└── Decision point: Approve or Reject?

Step 3a: Approval Path
├── RM Admin clicks "Approve"
├── Optional: Add approval notes
├── Confirm action
├── System updates:
│   ├── approval_status = 'approved'
│   ├── account_status = 'active'
│   ├── approved_by = admin_user_id
│   └── approved_at = current_timestamp
├── School admin user account activated
├── Email notification sent to school admin
└── School can now access main platform

Step 3b: Rejection Path
├── RM Admin clicks "Reject"
├── Required: Enter rejection reason
├── Confirm action
├── System updates:
│   ├── approval_status = 'rejected'
│   └── account_status = 'inactive'
├── Email notification sent to school admin with reason
└── School admin can resubmit with corrections
```

University Registration Workflow

```
Step 1: University Submits Registration
├── University admin fills registration form
├── Data includes: university info, colleges to be created
├── Status: approval_status = 'pending'
└── Notification sent to RM Admin
```

Step 2: RM Admin Reviews
├── Reviews university information
├── Reviews proposed college structure
└── Decision point: Approve or Reject?

Step 3: Approval
├── RM Admin approves university
├── System creates:
│   ├── University record (approved, active)
│   ├── University admin user account (activated)
│   └── Initial colleges (if provided)
├── Notifications sent
└── University admin can log in and manage colleges

Step 4: Post-Approval Management
├── University admin can create additional colleges
├── Each college can create courses
├── RM Admin can monitor all activities
└── RM Admin retains override control
```

### 10.2 Entity Suspension Workflow
```
Step 1: Decision to Suspend
├── RM Admin identifies issue with entity
├── Navigates to entity details page
├── Clicks "Suspend Entity"
└── Enters suspension reason (required)

Step 2: Confirmation
├── System shows "Are you sure?" dialog
├── Lists consequences:
│   ├── All users under this entity will be suspended
│   ├── No login access until reactivated
│   └── Data remains intact but inaccessible
└── RM Admin confirms

Step 3: System Actions

```
├── Updates entity: account_status = 'suspended'
├── Updates all users under entity: account_status = 'suspended'
├── Logs action in audit_logs
├── Sends notification emails to entity admin and all users
└── Immediate effect - all sessions invalidated

Step 4: Reactivation (when ready)
├── RM Admin clicks "Activate Entity"
├── Optional: Add reactivation notes
├── System restores access
└── Notification sent to entity admin
```

10.3 User Management Workflow

Suspend User Account

```
Step 1: Identify User
├── RM Admin searches for user
├── Views user details
└── Identifies reason for suspension

Step 2: Suspend Action
├── Clicks "Suspend User"
├── Enters suspension reason
├── Confirms action
└── System updates user: account_status = 'suspended'

Step 3: Effects
├── User cannot log in
├── Active sessions invalidated
├── Email notification sent to user
└── Audit log entry created

Step 4: Reactivation
├── RM Admin reviews case
├── Clicks "Activate User"
├── System restores access
└── User can log in again
```

```

#### Delete User Account
```

Step 1: Decision to Delete
├── RM Admin identifies user to delete
├── Reviews user's associated data
└── Understands consequences

Step 2: Delete Action
├── Clicks "Delete User"
├── System shows warning:
│   ├── This action is irreversible
│   ├── All user data will be permanently deleted
│   └── Related records will be affected
├── RM Admin confirms deletion
└── System performs cascading delete

Step 3: System Actions
├── Deletes user record from users table
├── Cascade delete or nullify related records:
│   ├── If educator: removes from educator table
│   ├── If student: removes from student table
│   ├── If recruiter: removes from recruiter table
│   └── Updates class/course counts
├── Logs action in audit_logs
└── No email sent (account deleted)

Step 4: Verification
├── User ID removed from all active sessions
├── User cannot be recovered
└── RM Admin sees confirmation message
```

### 10.4 Bulk Operations Workflow

#### Bulk Approval
```

Step 1: Select Entities

```
├── RM Admin goes to Approval Center
├── Filters pending entities (e.g., all pending schools)
├── Selects multiple entities using checkboxes
└── Clicks "Bulk Approve"

Step 2: Review Selection
├── System shows list of selected entities
├── RM Admin can review and deselect if needed
├── Optional: Add bulk approval notes
└── Confirms bulk action

Step 3: Processing
├── System shows progress indicator
├── For each entity:
│   ├── Updates approval_status = 'approved'
│   ├── Updates account_status = 'active'
│   ├── Activates admin user account
│   └── Sends notification email
├── Logs each action separately in audit_logs
└── Shows completion summary

Step 4: Results
├── Shows success count and any failures
├── Failed entities (if any) listed with reasons
└── RM Admin can retry failed ones individually
```

Bulk Rejection

```
Similar to bulk approval, but:
├── Requires single rejection reason for all
├── All selected entities get same reason
├── account_status = 'inactive'
└── Rejection emails sent to all
```

10.5 Analytics & Reporting Workflow

Generate Custom Report

```
Step 1: Report Configuration
├── RM Admin goes to Analytics > Reports
├── Selects report type:
│   ├── User Growth Report
│   ├── Entity Distribution Report
│   ├── Activity Report
│   └── Custom Report
└── Configures parameters

Step 2: Set Parameters
├── Date range selection
├── Entity type filters
├── Metrics to include
├── Grouping options (by date, location, etc.)
└── Export format (CSV, PDF, Excel)

Step 3: Generate Report
├── Clicks "Generate Report"
├── System queries database with filters
├── Aggregates data
├── Formats according to selected type
└── Shows preview

Step 4: Export/Download
├── RM Admin reviews preview
├── Clicks "Export"
├── System generates file
└── Downloads to admin's device

Step 5: Schedule Report (Optional)
├── RM Admin can schedule recurring reports
├── Sets frequency (daily, weekly, monthly)
├── Sets recipients (email addresses)
└── System sends automated reports
```

10.6 Audit Log Review Workflow

```
```

Step 1: Access Audit Logs
├── RM Admin goes to Audit Logs section
└── Sees all platform activities

Step 2: Filter & Search
├── Filters by:
│   ├── Date range
│   ├── User (who performed action)
│   ├── Action type (create, update, delete, etc.)
│   ├── Resource type (school, user, company, etc.)
│   └── Specific resource ID
└── Applies filters

Step 3: Review Entries
├── Views filtered audit log entries
├── Clicks on entry to see details:
│   ├── Who: User who performed action
│   ├── What: Action performed
│   ├── When: Timestamp
│   ├── Where: IP address, user agent
│   └── Changes: Old values vs new values (JSON diff)
└── Can drill down to related entries

Step 4: Export for Analysis
├── Selects date range for investigation
├── Exports audit logs as CSV
└── Can analyze in external tools

Step 5: Take Action (if needed)
├── If suspicious activity found:
│   ├── Suspend user
│   ├── Contact entity admin
│   └── Revert changes if necessary
└── Document findings
```

### 10.7 Real-time Notification Handling
```
Step 1: Notification Received

```
├── New entity registration
├── New user creation by entity admin
├── Suspicious activity alert
└── System errors or warnings

Step 2: Display Notification
├── Red badge on notification bell
├── Count of unread notifications
├── Sound/desktop notification (if enabled)
└── Toast message for critical alerts

Step 3: Review Notification
├── RM Admin clicks notification bell
├── Dropdown shows list of notifications
├── Each notification shows:
│   ├── Type icon
│   ├── Message summary
│   ├── Time ago
│   └── Quick action buttons
└── Can mark as read or click to view details

Step 4: Act on Notification
├── Click takes to relevant page
├── E.g., "New school pending approval" → Approval Center
├── Notification marked as read
└── Badge count decreases

Step 5: Notification Settings
├── RM Admin can configure:
│   ├── Which events trigger notifications
│   ├── Sound/desktop notification preferences
│   └── Email notification preferences
└── Saves preferences
```

---

11. Project Structure

```
```

```
rareminds-admin-app/
│
├── public/
│   ├── images/
│   │   ├── logo.svg
│   │   ├── illustrations/
│   │   └── icons/
│   └── favicon.ico
│
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   └── login/
│   │   │       ├── page.tsx
│   │   │       └── login-form.tsx
│   │   │
│   │   ├── (dashboard)/
│   │   │   ├── layout.tsx
│   │   │   ├── page.tsx
│   │   │   │
│   │   │   ├── schools/
│   │   │   │   ├── page.tsx
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   ├── classes/page.tsx
│   │   │   │   │   ├── educators/page.tsx
│   │   │   │   │   └── students/page.tsx
│   │   │   │   ├── create/page.tsx
│   │   │   │   └── components/
│   │   │   │       ├── school-table.tsx
│   │   │   │       ├── school-details-card.tsx
│   │   │   │       └── school-form.tsx
│   │   │   │
│   │   │   ├── colleges/
│   │   │   │   └── (similar structure)
│   │   │   │
│   │   │   ├── universities/
│   │   │   │   └── (similar structure)
│   │   │   │
```

```
│   │   ├── companies/
│   │   │   └── (similar structure)
│   │   │
│   │   ├── users/
│   │   │   ├── page.tsx
│   │   │   ├── [id]/page.tsx
│   │   │   └── components/
│   │   │
│   │   ├── approvals/
│   │   │   ├── page.tsx
│   │   │   ├── schools/page.tsx
│   │   │   ├── colleges/page.tsx
│   │   │   ├── universities/page.tsx
│   │   │   ├── companies/page.tsx
│   │   │   └── components/
│   │   │
│   │   ├── audit-logs/
│   │   │   ├── page.tsx
│   │   │   ├── [id]/page.tsx
│   │   │   └── components/
│   │   │
│   │   └── analytics/
│   │       ├── page.tsx
│   │       ├── users/page.tsx
│   │       ├── entities/page.tsx
│   │       ├── reports/page.tsx
│   │       └── components/
│   │
│   ├── api/ (if using Next.js API routes)
│   │
│   ├── globals.css
│   ├── layout.tsx
│   └── error.tsx
│
├── components/
│   ├── ui/ (shadcn/ui components)
│   │   ├── button.tsx
│   │   ├── dialog.tsx
│   │   ├── dropdown-menu.tsx
```

```
│   │   │   ├── table.tsx
│   │   │   ├── form.tsx
│   │   │   ├── input.tsx
│   │   │   ├── select.tsx
│   │   │   ├── badge.tsx
│   │   │   ├── card.tsx
│   │   │   ├── tabs.tsx
│   │   │   └── ... (other shadcn components)
│   │   │
│   │   ├── layout/
│   │   │   ├── sidebar.tsx
│   │   │   ├── header.tsx
│   │   │   ├── breadcrumb.tsx
│   │   │   └── dashboard-layout.tsx
│   │   │
│   │   ├── common/
│   │   │   ├── data-table.tsx
│   │   │   ├── search-bar.tsx
│   │   │   ├── status-badge.tsx
│   │   │   ├── confirm-dialog.tsx
│   │   │   ├── loading-state.tsx
│   │   │   ├── empty-state.tsx
│   │   │   ├── error-boundary.tsx
│   │   │   ├── file-uploader.tsx
│   │   │   ├── date-range-picker.tsx
│   │   │   └── notification-bell.tsx
│   │   │
│   │   ├── dashboard/
│   │   │   ├── stats-card.tsx
│   │   │   ├── entity-distribution-chart.tsx
│   │   │   ├── user-growth-chart.tsx
│   │   │   ├── recent-activity-feed.tsx
│   │   │   └── pending-approvals-list.tsx
│   │   │
│   │   └── forms/
│   │       ├── form-field.tsx
│   │       ├── form-section.tsx
│   │       ├── autocomplete-input.tsx
│   │       └── multi-select.tsx
```

```
│   │
│   ├── lib/
│   │   ├── api/
│   │   │   ├── client.ts
│   │   │   ├── endpoints.ts
│   │   │   └── queries/ (React Query hooks)
│   │   │       ├── use-schools.ts
│   │   │       ├── use-colleges.ts
│   │   │       ├── use-universities.ts
│   │   │       ├── use-companies.ts
│   │   │       ├── use-users.ts
│   │   │       ├── use-approvals.ts
│   │   │       └── use-audit-logs.ts
│   │   │
│   │   ├── auth/
│   │   │   ├── auth-context.tsx
│   │   │   ├── auth-provider.tsx
│   │   │   └── use-auth.ts
│   │   │
│   │   ├── utils/
│   │   │   ├── cn.ts (class name utility)
│   │   │   ├── format-date.ts
│   │   │   ├── format-currency.ts
│   │   │   ├── validators.ts
│   │   │   └── constants.ts
│   │   │
│   │   ├── stores/
│   │   │   ├── use-notification-store.ts
│   │   │   ├── use-sidebar-store.ts
│   │   │   └── use-theme-store.ts
│   │   │
│   │   └── websocket/
│   │       ├── websocket-client.ts
│   │       └── use-real-time-updates.ts
│   │
│   ├── types/
│   │   ├── entities.ts
│   │   ├── users.ts
│   │   ├── api.ts
```

```
│   │   ├── auth.ts
│   │   └── index.ts
│   │
│   ├── schemas/ (Zod schemas)
│   │   ├── school-schema.ts
│   │   ├── college-schema.ts
│   │   ├── university-schema.ts
│   │   ├── company-schema.ts
│   │   ├── user-schema.ts
│   │   └── index.ts
│   │
│   └── middleware.ts
│
├── .env.local
├── .env.production
├── .eslintrc.json
├── .prettierrc
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
├── package.json
├── pnpm-lock.yaml
└── README.md
```

---

12. Setup & Development Guide

12.1 Prerequisites

bash

```
# Required
Node.js 20+
pnpm (or npm/yarn)
Git

# Accounts needed
Supabase account (database)
Cloudflare account (hosting & API)
```

12.2 Initial Setup

bash

```
# 1. Clone repository
git clone https://github.com/rareminds/admin-app.git
cd admin-app

# 2. Install dependencies
pnpm install

# 3. Setup environment variables
cp .env.example .env.local

# Edit .env.local with your credentials
```

12.3 Environment Variables

.env.local:

bash

```
# Admin API
NEXT_PUBLIC_API_URL=http://localhost:8787
NEXT_PUBLIC_API_TIMEOUT=30000

# Supabase (for direct database access if needed)
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key

# App Config
NEXT_PUBLIC_APP_NAME=Rareminds Admin
NEXT_PUBLIC_APP_VERSION=1.0.0
NEXT_PUBLIC_ENVIRONMENT=development

# WebSocket (for real-time updates)
NEXT_PUBLIC_WS_URL=ws://localhost:8788

# Feature Flags
NEXT_PUBLIC_ENABLE_ANALYTICS=true
NEXT_PUBLIC_ENABLE_NOTIFICATIONS=true
```

.env.production:

bash

```
NEXT_PUBLIC_API_URL=https://admin-api.rareminds.com
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
NEXT_PUBLIC_WS_URL=wss://admin-ws.rareminds.com
NEXT_PUBLIC_ENVIRONMENT=production
```

12.4 Development Commands

bash

```
# Start development server
pnpm dev
# App runs on http://localhost:3000

# Build for production
pnpm build

# Start production server (locally)
pnpm start

# Lint code
pnpm lint

# Format code
pnpm format

# Type check
pnpm type-check

# Run tests
pnpm test

# Run E2E tests
pnpm test:e2e
```

12.5 Database Setup

bash

```
# If you need to run database migrations or seeds locally

# Install Supabase CLI
brew install supabase/tap/supabase
```

```
# Link to your Supabase project
supabase link --project-ref your-project-ref

# Pull remote schema
supabase db pull

# Or push local schema
supabase db push

# Run seeds
supabase db seed
```

12.6 Admin API Setup (Backend)

The Admin App requires the Admin API (Backend API 1) to be running.

bash

```
# In a separate terminal, navigate to admin-api directory
cd ../admin-api

# Install dependencies
pnpm install

# Setup environment variables
cp .env.example .env

# Edit .env with your Supabase credentials and JWT secret

# Start development server
pnpm dev
# API runs on http://localhost:8787
```

12.7 First Time Setup - Create Admin User

After setting up the database and APIs, you need to create the first admin user.

bash

```
# Option 1: Using Supabase Dashboard
# 1. Go to Supabase Dashboard > Authentication > Users
# 2. Click "Add user" > "Create new user"
# 3. Enter email and password
```

```
# 4. Copy the user ID

# 2. Go to SQL Editor and run:
INSERT INTO users (
  supabase_auth_id,
  email,
  first_name,
  last_name,
  role,
  account_status,
  entity_type,
  entity_id
) VALUES (
  'paste-user-id-here',
  'admin@rareminds.com',
  'Admin',
  'User',
  'platform_admin',
  'active',
  NULL,
  NULL
);

# Option 2: Using SQL Script (recommended)
# Create a seed file: database/seeds/001_create_admin.sql

BEGIN;

-- Create admin user in Supabase Auth (you'll need to do this via Supabase Dashboard or API)
-- Then insert into users table

INSERT INTO users (
  supabase_auth_id,
  email,
  first_name,
  last_name,
  role,
  account_status
) VALUES (
```

```
  'your-supabase-auth-id',
  'admin@rareminds.com',
  'Platform',
  'Admin',
  'platform_admin',
  'active'
);

COMMIT;
```

12.8 Development Workflow

```
Day-to-Day Development:

1. Pull latest changes
   git pull origin main

2. Install new dependencies (if any)
   pnpm install

3. Start both Admin App and Admin API
   Terminal 1: cd admin-app && pnpm dev
   Terminal 2: cd admin-api && pnpm dev

4. Make changes to code
   - Frontend changes auto-reload
   - Backend changes require restart

5. Test changes
   - Manual testing in browser
   - Run unit tests: pnpm test
   - Run E2E tests: pnpm test:e2e

6. Commit changes
   git add .
   git commit -m "feat: add feature X"
   git push origin feature-branch
```

7. Create Pull Request
   - Request review from team
   - CI/CD runs automated tests
   - Merge after approval

12.9 Common Development Tasks

Add a new entity type to manage:

typescript

```typescript
// 1. Add types in src/types/entities.ts
export interface NewEntity {
  id: string;
  name: string;
  // ... other fields
}

// 2. Add API endpoints in src/lib/api/endpoints.ts
export const newEntityEndpoints = {
  list: '/api/admin/new-entities',
  get: (id: string) => `/api/admin/new-entities/${id}`,
  create: '/api/admin/new-entities',
  update: (id: string) => `/api/admin/new-entities/${id}`,
  delete: (id: string) => `/api/admin/new-entities/${id}`,
};

// 3. Create React Query hooks in src/lib/api/queries/use-new-entities.ts
export function useNewEntities() {
  return useInfiniteQuery({
    queryKey: ['new-entities'],
    queryFn: ({ pageParam = 0 }) =>
      apiClient.get(newEntityEndpoints.list, {
        params: { offset: pageParam, limit: 20 }
      }),
    getNextPageParam: (lastPage) => lastPage.nextOffset,
  });
}

// 4. Create pages in src/app/(dashboard)/new-entities/
// 5. Create components in src/app/(dashboard)/new-entities/components/
```

```
// 6. Add navigation link in sidebar component
```

Add a new permission:

sql

```
-- In database
INSERT INTO permissions (name, resource, action, description)
VALUES ('new_entity:manage', 'new_entity', 'manage', 'Manage new entities');

-- Assign to platform_admin
INSERT INTO role_permissions (role, permission_id)
SELECT 'platform_admin', id FROM permissions WHERE name = 'new_entity:manage';
```

Add a new chart to dashboard:

typescript

```
// 1. Create component in src/components/dashboard/new-chart.tsx
export function NewChart() {
 const { data } = useNewChartData();

 return (
  <Card>
   <CardHeader>
    <CardTitle>New Chart Title</CardTitle>
   </CardHeader>
   <CardContent>
    <ResponsiveContainer width="100%" height={300}>
     <LineChart data={data}>
      {/* Chart configuration */}
     </LineChart>
    </ResponsiveContainer>
   </CardContent>
  </Card>
 );
}

// 2. Add to dashboard page
// src/app/(dashboard)/page.tsx
import { NewChart } from '@/components/dashboard/new-chart';

export default function DashboardPage() {
```

```
  return (
    <div className="grid gap-4">
      {/* Existing charts */}
      <NewChart />
    </div>
  );
}
```

---

13. Deployment Guide

13.1 Pre-Deployment Checklist

bash

```
# 1. Run all tests
pnpm test
pnpm test:e2e

# 2. Type check
pnpm type-check

# 3. Lint code
pnpm lint

# 4. Build locally to check for errors
pnpm build

# 5. Test production build locally
pnpm start

# 6. Check environment variables
# Ensure all production env vars are set in Cloudflare Pages

# 7. Database migrations
# Ensure all migrations are applied to production database
supabase db push --project-ref production-project

# 8. Backend API deployed first
# Admin App depends on Admin API, so deploy API first
```

13.2 Cloudflare Pages Deployment

Initial Setup

bash

```
# 1. Install Wrangler CLI
npm install -g wrangler

# 2. Login to Cloudflare
wrangler login

# 3. Create Cloudflare Pages project
wrangler pages project create rareminds-admin

# 4. Link local project to Cloudflare Pages
# Create wrangler.toml in project root

# wrangler.toml
name = "rareminds-admin"
compatibility_date = "2024-01-01"

[env.production]
vars = { ENVIRONMENT = "production" }

[env.staging]
vars = { ENVIRONMENT = "staging" }
```

Manual Deployment

bash

```
# 1. Build the app
pnpm build

# 2. Deploy to Cloudflare Pages
wrangler pages deploy .next --project-name=rareminds-admin --branch=main

# For staging
wrangler pages deploy .next --project-name=rareminds-admin --branch=staging
```

Configure Custom Domain

bash

```
# Via Wrangler CLI
wrangler pages domains add admin.rareminds.com --project-name=rareminds-admin

# Or via Cloudflare Dashboard:
# Pages > rareminds-admin > Custom domains > Add domain
```

13.3 CI/CD with GitHub Actions

Create `.github/workflows/deploy-admin.yml`:

yaml

```yaml
name: Deploy Admin App

on:
  push:
    branches:
      - main
      - staging
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install pnpm
        uses: pnpm/action-setup@v2
        with:
          version: 8

      - name: Install dependencies
        run: pnpm install
```

```yaml
    - name: Type check
      run: pnpm type-check

    - name: Lint
      run: pnpm lint

    - name: Run tests
      run: pnpm test

    - name: Build
      run: pnpm build
      env:
        NEXT_PUBLIC_API_URL: ${{ secrets.API_URL }}
        NEXT_PUBLIC_SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
        NEXT_PUBLIC_SUPABASE_ANON_KEY: ${{ secrets.SUPABASE_ANON_KEY }}

  deploy-staging:
    needs: test
    if: github.ref == 'refs/heads/staging'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install pnpm
        uses: pnpm/action-setup@v2
        with:
          version: 8

      - name: Install dependencies
        run: pnpm install

      - name: Build
        run: pnpm build
```

```yaml
      env:
        NEXT_PUBLIC_API_URL: ${{ secrets.STAGING_API_URL }}
        NEXT_PUBLIC_SUPABASE_URL: ${{ secrets.STAGING_SUPABASE_URL }}
        NEXT_PUBLIC_SUPABASE_ANON_KEY: ${{ secrets.STAGING_SUPABASE_ANON_KEY }}
        NEXT_PUBLIC_ENVIRONMENT: staging

      - name: Deploy to Cloudflare Pages (Staging)
        uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
          accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
          command: pages deploy .next --project-name=rareminds-admin --branch=staging

  deploy-production:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install pnpm
        uses: pnpm/action-setup@v2
        with:
          version: 8

      - name: Install dependencies
        run: pnpm install

      - name: Build
        run: pnpm build
        env:
          NEXT_PUBLIC_API_URL: ${{ secrets.PRODUCTION_API_URL }}
          NEXT_PUBLIC_SUPABASE_URL: ${{ secrets.PRODUCTION_SUPABASE_URL }}
```

```
      NEXT_PUBLIC_SUPABASE_ANON_KEY: ${{ secrets.PRODUCTION_SUPABASE_ANON_KEY }}
      NEXT_PUBLIC_ENVIRONMENT: production

    - name: Deploy to Cloudflare Pages (Production)
      uses: cloudflare/wrangler-action@v3
      with:
        apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
        accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
        command: pages deploy .next --project-name=rareminds-admin --branch=main

    - name: Notify deployment
      if: success()
      run: |
        echo "Deployment successful to admin.rareminds.com"
        # Add Slack/Discord notification here if needed
```

13.4 Environment Variables in Cloudflare

Set environment variables via Cloudflare Dashboard or Wrangler:

bash

```
# Via Wrangler CLI
wrangler pages secret put NEXT_PUBLIC_API_URL --project-name=rareminds-admin
# Enter value when prompted

# Or via Cloudflare Dashboard:
# Pages > rareminds-admin > Settings > Environment variables
# Add variables for both Production and Preview
```

Production Environment Variables:

```
NEXT_PUBLIC_API_URL=https://admin-api.rareminds.com
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-production-anon-key
NEXT_PUBLIC_WS_URL=wss://admin-ws.rareminds.com
NEXT_PUBLIC_ENVIRONMENT=production
NEXT_PUBLIC_ENABLE_ANALYTICS=true
NEXT_PUBLIC_ENABLE_NOTIFICATIONS=true
```

Staging Environment Variables:

```
NEXT_PUBLIC_API_URL=https://admin-api-staging.rareminds.com
NEXT_PUBLIC_SUPABASE_URL=https://your-staging-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-staging-anon-key
NEXT_PUBLIC_WS_URL=wss://admin-ws-staging.rareminds.com
NEXT_PUBLIC_ENVIRONMENT=staging
```

13.5 Post-Deployment Verification

bash

```
# 1. Check deployment status
wrangler pages deployment list --project-name=rareminds-admin

# 2. Smoke tests
curl https://admin.rareminds.com
# Should return HTML (status 200)

curl https://admin.rareminds.com/api/health
# Should return health check response (if you have one)

# 3. Login test
# Open browser, navigate to admin.rareminds.com
# Login with admin credentials
# Verify dashboard loads correctly

# 4. Check real-time notifications
# Create a test entity in main platform
# Verify notification appears in admin app

# 5. Check analytics
# Navigate to Analytics section
# Verify charts load correctly

# 6. Monitor logs
# Check Cloudflare Pages logs for any errors
wrangler pages deployment tail --project-name=rareminds-admin
```

13.6 Rollback Procedure

bash

```
# If deployment has issues, rollback to previous version

# 1. List recent deployments
wrangler pages deployment list --project-name=rareminds-admin

# 2. Identify previous working deployment ID
# Example: 1234abcd-5678-efgh-9012-ijklmnopqrst

# 3. Promote previous deployment to production
wrangler pages deployment promote 1234abcd-5678-efgh-9012-ijklmnopqrst \
  --project-name=rareminds-admin

# 4. Verify rollback
curl https://admin.rareminds.com

# 5. Fix issues in code, then redeploy
```

13.7 Blue-Green Deployment Strategy

bash

```
# For zero-downtime deployments

# 1. Deploy to staging first
git push origin staging

# 2. Test thoroughly on staging
# Run automated tests, manual QA

# 3. Once validated, promote staging to production
# This happens automatically when you merge to main
git checkout main
git merge staging
git push origin main

# 4. Cloudflare Pages automatically deploys
# Old version still serves traffic until new version is ready

# 5. Once new deployment is live, traffic switches
# No downtime experienced by users
```

---

14. Security Considerations

14.1 Authentication Security

JWT Token Security:

typescript

```typescript
// Token configuration
const TOKEN_CONFIG = {
  accessTokenExpiry: '24h',
  refreshTokenExpiry: '7d',
  algorithm: 'HS256',
  issuer: 'rareminds-admin-api',
  audience: 'rareminds-admin-app',
};

// Token storage
// ☑ Correct: httpOnly cookies
document.cookie = `admin_token=${token}; HttpOnly; Secure; SameSite=Strict`;

// ✗ Wrong: localStorage (vulnerable to XSS)
// localStorage.setItem('admin_token', token);
```

Session Management:

typescript

```typescript
// Implement session timeout
const SESSION_TIMEOUT = 24 * 60 * 60 * 1000; // 24 hours
const IDLE_TIMEOUT = 30 * 60 * 1000; // 30 minutes

let lastActivity = Date.now();

// Track user activity
function trackActivity() {
  lastActivity = Date.now();
}

// Check for idle timeout
setInterval(() => {
```

```
  const idleTime = Date.now() - lastActivity;
  if (idleTime > IDLE_TIMEOUT) {
    showIdleWarning();
  }
}, 60000); // Check every minute

function showIdleWarning() {
  // Show modal: "You will be logged out in 5 minutes"
  // Give option to extend session
}
```

Password Requirements:

typescript

```
// Password validation schema
const passwordSchema = z
  .string()
  .min(8, 'Password must be at least 8 characters')
  .regex(/[A-Z]/, 'Password must contain at least one uppercase letter')
  .regex(/[a-z]/, 'Password must contain at least one lowercase letter')
  .regex(/[0-9]/, 'Password must contain at least one number')
  .regex(/[^A-Za-z0-9]/, 'Password must contain at least one special character');
```

14.2 Authorization Security

Role-Based Access Control:

typescript

```
// Verify user role on every protected route
export function requirePlatformAdmin(user: User) {
  if (user.role !== 'platform_admin') {
    throw new Error('Unauthorized: Platform admin access required');
  }

  if (user.account_status !== 'active') {
    throw new Error('Account is not active');
  }
}

// Check specific permissions
export function requirePermission(user: User, permission: string) {
```

```
  if (!user.permissions.includes(permission) &&
      !user.permissions.includes('platform:manage_all')) {
    throw new Error(`Missing permission: ${permission}`);
  }
}
```

API Request Authorization:

typescript

```
// Every API request must include valid JWT
async function makeAuthorizedRequest(endpoint: string, options: RequestInit = {}) {
  const token = getCookie('admin_token');

  if (!token) {
    throw new Error('No authentication token found');
  }

  const response = await fetch(`${API_BASE_URL}${endpoint}`, {
    ...options,
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
      ...options.headers,
    },
  });

  if (response.status === 401) {
    // Token expired or invalid
    await handleTokenExpiry();
    throw new Error('Authentication expired');
  }

  if (response.status === 403) {
    throw new Error('Insufficient permissions');
  }

  return response;
}
```

14.3 Data Security

Input Validation:

typescript

```typescript
// Always validate input on both client and server
import { z } from 'zod';

// Example: School creation validation
const schoolSchema = z.object({
  name: z.string().min(2).max(255),
  code: z.string().regex(/^[A-Z0-9]{3,10}$/),
  email: z.string().email(),
  phone: z.string().regex(/^\+?[1-9]\d{1,14}$/),
  address: z.string().min(5),
  city: z.string().min(2),
  state: z.string().min(2),
  pincode: z.string().regex(/^\d{6}$/),
});

// Validate before sending to API
function validateSchoolData(data: unknown) {
  try {
    return schoolSchema.parse(data);
  } catch (error) {
    if (error instanceof z.ZodError) {
      // Show validation errors to user
      showValidationErrors(error.errors);
    }
    throw error;
  }
}
```

SQL Injection Prevention:

typescript

```typescript
// ☑ Correct: Use parameterized queries (in backend API)
const { data } = await supabase
  .from('schools')
  .select('*')
  .eq('id', schoolId); // Safe - parameterized
```

```
// ❌ Wrong: String concatenation (vulnerable)
// const query = `SELECT * FROM schools WHERE id = '${schoolId}'`;
```

XSS Prevention:

typescript

```
// React automatically escapes content
// But be careful with dangerouslySetInnerHTML

// ☑ Correct: Let React handle escaping
<div>{userInput}</div>

// ❌ Dangerous: Only use if you sanitize first
// <div dangerouslySetInnerHTML={{ __html: userInput }} />

// If you must render HTML, sanitize it first
import DOMPurify from 'dompurify';

function SafeHTML({ html }: { html: string }) {
 const sanitized = DOMPurify.sanitize(html);
 return <div dangerouslySetInnerHTML={{ __html: sanitized }} />;
}
```

14.4 API Security

Rate Limiting:

typescript

```
// Implemented in Backend API (Cloudflare Worker)
// Admin API should have higher rate limits than main platform

const RATE_LIMITS = {
 login: {
  requests: 5,
  window: 15 * 60 * 1000, // 15 minutes
 },
 api: {
  requests: 1000,
  window: 60 * 1000, // 1 minute
 },
};
```

CORS Configuration:

typescript

```typescript
// Backend API CORS settings
const corsConfig = {
  origin: [
    'https://admin.rareminds.com',
    'https://admin-staging.rareminds.com',
  ],
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
  credentials: true,
  allowedHeaders: ['Content-Type', 'Authorization'],
};
```

API Request Signing (Optional - Extra Security):

typescript

```typescript
// For critical operations, sign requests with HMAC
import { createHmac } from 'crypto';

function signRequest(data: any, secret: string): string {
  const timestamp = Date.now();
  const payload = JSON.stringify({ ...data, timestamp });
  const signature = createHmac('sha256', secret)
    .update(payload)
    .digest('hex');
  return signature;
}

// Backend verifies signature
function verifySignature(data: any, signature: string, secret: string): boolean {
  const expectedSignature = signRequest(data, secret);
  return signature === expectedSignature;
}
```

14.5 Sensitive Data Handling

Logging:

typescript

```typescript
// Never log sensitive information
// ☑ Correct
```

```
console.log('User logged in', { userId: user.id, email: user.email });

// ✕ Wrong
// console.log('User logged in', { password: user.password });
```

Audit Logging:

typescript

```
// Log all sensitive operations
async function logSensitiveAction(
  action: string,
  userId: string,
  details: Record<string, any>
) {
  await supabase.from('audit_logs').insert({
    user_id: userId,
    action,
    resource_type: 'sensitive',
    old_values: null,
    new_values: details,
    ip_address: getClientIP(),
    user_agent: getUserAgent(),
  });
}

// Example usage
await logSensitiveAction('PASSWORD_RESET', userId, {
  target_user: targetUserId,
  reset_method: 'admin_initiated',
});
```

Data Encryption:

typescript

```
// Encrypt sensitive data before storing
import { encrypt, decrypt } from '@/lib/encryption';

// Example: Storing API keys
async function storeAPIKey(entityId: string, apiKey: string) {
  const encrypted = encrypt(apiKey, process.env.ENCRYPTION_KEY!);
```

```
  await supabase
    .from('entity_api_keys')
    .insert({
      entity_id: entityId,
      encrypted_key: encrypted,
    });
}

// Decrypt when needed
async function getAPIKey(entityId: string): Promise<string> {
  const { data } = await supabase
    .from('entity_api_keys')
    .select('encrypted_key')
    .eq('entity_id', entityId)
    .single();

  return decrypt(data.encrypted_key, process.env.ENCRYPTION_KEY!);
}
```

14.6 Security Headers

Next.js Configuration:

javascript

```
// next.config.js
module.exports = {
  async headers() {
    return [
      {
        source: '/:path*',
        headers: [
          {
            key: 'X-DNS-Prefetch-Control',
            value: 'on'
          },
          {
            key: 'Strict-Transport-Security',
            value: 'max-age=63072000; includeSubDomains; preload'
          },
          {
```

```
          key: 'X-Frame-Options',
          value: 'DENY'
        },
        {
          key: 'X-Content-Type-Options',
          value: 'nosniff'
        },
        {
          key: 'X-XSS-Protection',
          value: '1; mode=block'
        },
        {
          key: 'Referrer-Policy',
          value: 'strict-origin-when-cross-origin'
        },
        {
          key: 'Permissions-Policy',
          value: 'camera=(), microphone=(), geolocation=()'
        },
        {
          key: 'Content-Security-Policy',
          value: `
            default-src 'self';
            script-src 'self' 'unsafe-eval' 'unsafe-inline';
            style-src 'self' 'unsafe-inline';
            img-src 'self' data: https:;
            font-src 'self' data:;
            connect-src 'self' https://admin-api.rareminds.com https://*.supabase.co wss://*.supabase.co;
            frame-ancestors 'none';
          `.replace(/\s{2,}/g, ' ').trim()
        }
      ]
    }
  ];
 }
};
```

14.7 Dependency Security

bash

 # Regular security audits

pnpm audit

 # Fix vulnerabilities automatically

pnpm audit --fix

 # Check for outdated packages

pnpm outdated

 # Update dependencies

pnpm update

 # Use Snyk or Dependabot for automated security alerts

 # Configure in GitHub repository settings

```

*### 14.8 Secure Deployment Checklist*

```
☐ All environment variables set correctly

☐ No secrets in code or version control

☐ HTTPS enabled (Cloudflare does this automatically)

☐ Security headers configured

☐ CORS properly configured

☐ Rate limiting enabled

☐ Authentication working correctly

☐ Authorization checks in place

☐ Input validation on all forms

☐ SQL injection protection (parameterized queries)

☐ XSS protection (React escaping + DOMPurify if needed)

☐ CSRF protection

☐ Audit logging enabled

☐ Error logging (without sensitive data)

☐ Regular security updates scheduled

☐ Backup and disaster recovery plan in place

---

15. Monitoring & Analytics

15.1 Application Monitoring

Cloudflare Analytics:

Cloudflare Pages automatically provides:

- Page views and unique visitors
- Requests by country
- Response status codes
- Bandwidth usage
- Core Web Vitals

Access via: Cloudflare Dashboard > Pages > rareminds-admin > Analytics

Custom Monitoring:

typescript

```typescript
// Track custom events
export function trackEvent(eventName: string, properties?: Record<string, any>) {
 // Send to analytics service (e.g., Plausible, PostHog, or custom)
 if (typeof window !== 'undefined' && window.plausible) {
  window.plausible(eventName, { props: properties });
 }
}

// Usage examples
trackEvent('entity_approved', { entityType: 'school', entityId: schoolId });
trackEvent('user_suspended', { userId, reason });
trackEvent('bulk_action', { action: 'approve', count: 10 });
```

15.2 Error Monitoring

Sentry Integration:

bash

```bash
# Install Sentry
pnpm add @sentry/nextjs
```

typescript

```typescript
// sentry.client.config.ts
import * as Sentry from '@sentry/nextjs';

Sentry.init({
 dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
 environment: process.env.NEXT_PUBLIC_ENVIRONMENT,
```

```
  tracesSampleRate: 1.0,

  // Don't send sensitive data
  beforeSend(event, hint) {
    // Filter out sensitive information
    if (event.request) {
      delete event.request.cookies;
      delete event.request.headers;
    }
    return event;
  },

  ignoreErrors: [
    // Ignore common non-critical errors
    'ResizeObserver loop limit exceeded',
    'Non-Error promise rejection captured',
  ],
});
```

typescript

```
// Error boundary with Sentry
import * as Sentry from '@sentry/nextjs';
import { ErrorBoundary as SentryErrorBoundary } from '@sentry/nextjs';

export function ErrorBoundary({ children }: { children: React.ReactNode }) {
  return (
    <SentryErrorBoundary
      fallback={({ error, resetError }) => (
        <div className="flex flex-col items-center justify-center min-h-screen">
          <h1 className="text-2xl font-bold mb-4">Something went wrong</h1>
          <p className="text-gray-600 mb-4">{error.message}</p>
          <button onClick={resetError} className="btn btn-primary">
            Try again
          </button>
        </div>
      )}
    >
      {children}
    </SentryErrorBoundary>
  );
```

```
}
```

15.3 Performance Monitoring

Web Vitals Tracking:

typescript

```typescript
// pages/_app.tsx or app/layout.tsx
import { useReportWebVitals } from 'next/web-vitals';

export default function App({ Component, pageProps }: AppProps) {
  useReportWebVitals((metric) => {
    // Send to analytics
    trackEvent('web_vital', {
      metric: metric.name,
      value: metric.value,
      rating: metric.rating,
    });

    // Also send to custom endpoint
    fetch('/api/analytics/web-vitals', {
      method: 'POST',
      body: JSON.stringify(metric),
    });
  });

  return <Component {...pageProps} />;
}
```

API Performance Tracking:

typescript

```typescript
// Wrap API calls with timing
async function makeTimedRequest(endpoint: string, options?: RequestInit) {
  const startTime = performance.now();

  try {
    const response = await fetch(endpoint, options);
    const endTime = performance.now();
    const duration = endTime - startTime;
```

```
    // Log performance
    trackEvent('api_request', {
      endpoint,
      duration,
      status: response.status,
    });

    return response;
  } catch (error) {
    const endTime = performance.now();
    const duration = endTime - startTime;

    // Log failed request
    trackEvent('api_error', {
      endpoint,
      duration,
      error: error.message,
    });

    throw error;
  }
}
```

15.4 User Activity Analytics

Track Key Actions:

typescript

```
// Track important admin actions (continued)
const TRACKED_ACTIONS = {
  // Entity management
  ENTITY_APPROVED: 'entity_approved',
  ENTITY_REJECTED: 'entity_rejected',
  ENTITY_SUSPENDED: 'entity_suspended',
  ENTITY_DELETED: 'entity_deleted',

  // User management
  USER_SUSPENDED: 'user_suspended',
  USER_ACTIVATED: 'user_activated',
  USER_DELETED: 'user_deleted',
```

```
  PASSWORD_RESET: 'password_reset',

  // Bulk actions
  BULK_APPROVAL: 'bulk_approval',
  BULK_REJECTION: 'bulk_rejection',

  // Reports
  REPORT_GENERATED: 'report_generated',
  AUDIT_LOG_EXPORTED: 'audit_log_exported',

  // System
  LOGIN: 'admin_login',
  LOGOUT: 'admin_logout',
};

// Track function
export function trackAdminAction(
  action: string,
  properties?: Record<string, any>
) {
  // Send to analytics
  trackEvent(action, {
    ...properties,
    timestamp: new Date().toISOString(),
    admin_id: getCurrentUserId(),
  });

  // Also log to audit trail (via API)
  logAuditTrail(action, properties);
}

// Usage
trackAdminAction(TRACKED_ACTIONS.ENTITY_APPROVED, {
  entityType: 'school',
  entityId: school.id,
  entityName: school.name,
});
```

Analytics Dashboard Queries:

typescript

```typescript
// Example: Get most active admins
export async function getMostActiveAdmins(dateRange: DateRange) {
  const { data } = await supabase
    .from('audit_logs')
    .select('user_id, users(first_name, last_name, email), count')
    .gte('created_at', dateRange.from)
    .lte('created_at', dateRange.to)
    .order('count', { ascending: false })
    .limit(10);

  return data;
}

// Example: Get most common actions
export async function getMostCommonActions(dateRange: DateRange) {
  const { data } = await supabase
    .from('audit_logs')
    .select('action, count')
    .gte('created_at', dateRange.from)
    .lte('created_at', dateRange.to)
    .order('count', { ascending: false });

  return data;
}

// Example: Get entity approval metrics
export async function getApprovalMetrics(dateRange: DateRange) {
  const { data } = await supabase.rpc('get_approval_metrics', {
    date_from: dateRange.from,
    date_to: dateRange.to,
  });

  return data;
}
```

15.5 Real-Time Monitoring

WebSocket Connection for Live Updates:

typescript

```typescript
// lib/websocket/websocket-client.ts
export class AdminWebSocketClient {
  private ws: WebSocket | null = null;
  private reconnectAttempts = 0;
  private maxReconnectAttempts = 5;

  connect(token: string) {
    const wsUrl = `${process.env.NEXT_PUBLIC_WS_URL}?token=${token}`;

    this.ws = new WebSocket(wsUrl);

    this.ws.onopen = () => {
      console.log('WebSocket connected');
      this.reconnectAttempts = 0;
    };

    this.ws.onmessage = (event) => {
      const data = JSON.parse(event.data);
      this.handleMessage(data);
    };

    this.ws.onerror = (error) => {
      console.error('WebSocket error:', error);
    };

    this.ws.onclose = () => {
      console.log('WebSocket disconnected');
      this.reconnect(token);
    };
  }

  private reconnect(token: string) {
    if (this.reconnectAttempts < this.maxReconnectAttempts) {
      this.reconnectAttempts++;
      setTimeout(() => {
        console.log(`Reconnecting... (${this.reconnectAttempts})`);
        this.connect(token);
      }, 1000 * this.reconnectAttempts);
```

```
  }
}

private handleMessage(data: any) {
  switch (data.type) {
    case 'new_entity_pending':
      this.onNewEntityPending(data.payload);
      break;
    case 'entity_approved':
      this.onEntityApproved(data.payload);
      break;
    case 'user_created':
      this.onUserCreated(data.payload);
      break;
    case 'system_alert':
      this.onSystemAlert(data.payload);
      break;
    default:
      console.log('Unknown message type:', data.type);
  }
}

private onNewEntityPending(payload: any) {
  // Update pending approvals count
  // Show notification
  toast.info(`New ${payload.entityType} pending approval: ${payload.entityName}`);
}

private onEntityApproved(payload: any) {
  // Update entity list if visible
  // Update stats
}

private onUserCreated(payload: any) {
  // Update user count
}

private onSystemAlert(payload: any) {
  // Show critical alert
```

```
    toast.error(payload.message);
  }

  disconnect() {
    if (this.ws) {
      this.ws.close();
      this.ws = null;
    }
  }
}

// Hook for using WebSocket
export function useAdminWebSocket() {
  const [client] = useState(() => new AdminWebSocketClient());
  const { token } = useAuth();

  useEffect(() => {
    if (token) {
      client.connect(token);
    }

    return () => {
      client.disconnect();
    };
  }, [token, client]);

  return client;
}
```

15.6 System Health Monitoring

Health Check Endpoint (Backend API):

typescript

```
// Backend API health check
router.get('/health', async (c) => {
  const checks = {
    api: 'ok',
    database: 'unknown',
    timestamp: new Date().toISOString(),
```

```
};

// Check database connection
try {
  const { error } = await c.env.supabase
    .from('users')
    .select('count')
    .limit(1);

  checks.database = error ? 'error' : 'ok';
} catch (error) {
  checks.database = 'error';
}

const allHealthy = Object.values(checks).every(
  status => status === 'ok' || status === 'unknown'
);

return c.json(checks, allHealthy ? 200 : 503);
});
```

Frontend Health Monitoring:

typescript

```
// Regular health checks
export function useSystemHealth() {
  const { data: health, isError } = useQuery({
    queryKey: ['system-health'],
    queryFn: async () => {
      const response = await fetch(`${API_BASE_URL}/health`);
      return response.json();
    },
    refetchInterval: 60000, // Check every minute
    retry: 3,
  });

  useEffect(() => {
    if (isError) {
      // Show system alert
      toast.error('System health check failed. Some features may be unavailable.');
```

```
  }
 }, [isError]);

 return health;
}
```

15.7 Uptime Monitoring

External Monitoring Services:

bash

```
# Recommended services for uptime monitoring:

1. UptimeRobot (free tier available)
   - Monitor: https://admin.rareminds.com
   - Monitor: https://admin-api.rareminds.com/health
   - Alert via: Email, SMS, Slack

2. Pingdom
   - More detailed performance monitoring
   - Global monitoring locations

3. Better Uptime
   - Status page
   - Incident management

4. Cloudflare Analytics (included)
   - Built-in monitoring
   - DDoS protection
```

15.8 Log Management

Structured Logging:

typescript

```
// lib/logger.ts
enum LogLevel {
 DEBUG = 'debug',
 INFO = 'info',
 WARN = 'warn',
 ERROR = 'error',
}
```

```typescript
interface LogEntry {
  level: LogLevel;
  message: string;
  timestamp: string;
  context?: Record<string, any>;
  userId?: string;
  requestId?: string;
}

export class Logger {
  private static instance: Logger;

  private constructor() {}

  static getInstance(): Logger {
    if (!Logger.instance) {
      Logger.instance = new Logger();
    }
    return Logger.instance;
  }

  private log(level: LogLevel, message: string, context?: Record<string, any>) {
    const entry: LogEntry = {
      level,
      message,
      timestamp: new Date().toISOString(),
      context,
      userId: getCurrentUserId(),
      requestId: getRequestId(),
    };

    // Send to logging service (e.g., Datadog, LogRocket)
    if (process.env.NODE_ENV === 'production') {
      this.sendToLoggingService(entry);
    } else {
      console.log(JSON.stringify(entry, null, 2));
    }
  }
```

```
  private async sendToLoggingService(entry: LogEntry) {
    // Send to external logging service
    // Example: Datadog, LogRocket, Sentry
    try {
      await fetch('/api/logs', {
        method: 'POST',
        body: JSON.stringify(entry),
      });
    } catch (error) {
      // Fallback to console if logging service fails
      console.error('Failed to send log:', error);
    }
  }

  debug(message: string, context?: Record<string, any>) {
    this.log(LogLevel.DEBUG, message, context);
  }

  info(message: string, context?: Record<string, any>) {
    this.log(LogLevel.INFO, message, context);
  }

  warn(message: string, context?: Record<string, any>) {
    this.log(LogLevel.WARN, message, context);
  }

  error(message: string, context?: Record<string, any>) {
    this.log(LogLevel.ERROR, message, context);
  }
}

export const logger = Logger.getInstance();

// Usage
logger.info('Entity approved', {
  entityType: 'school',
  entityId: school.id,
  approvedBy: admin.id,
```

```
});

logger.error('Failed to suspend user', {
  userId: user.id,
  error: error.message,
  stack: error.stack,
});
```

15.9 Alerting System

Critical Alerts Configuration:

typescript

```typescript
// lib/alerts.ts
interface AlertConfig {
  type: 'email' | 'sms' | 'slack' | 'webhook';
  enabled: boolean;
  recipients: string[];
}

interface AlertRule {
  name: string;
  condition: () => Promise<boolean>;
  severity: 'low' | 'medium' | 'high' | 'critical';
  message: string;
  config: AlertConfig;
}

export const alertRules: AlertRule[] = [
  {
    name: 'High Error Rate',
    condition: async () => {
      const errorRate = await getErrorRate();
      return errorRate > 0.05; // 5% error rate
    },
    severity: 'critical',
    message: 'Error rate exceeded 5% in the last 5 minutes',
    config: {
      type: 'email',
      enabled: true,
```

```
    recipients: ['admin@rareminds.com', 'tech@rareminds.com'],
   },
  },
  {
   name: 'Database Connection Issues',
   condition: async () => {
    try {
     const health = await checkDatabaseHealth();
     return !health.ok;
    } catch {
     return true;
    }
   },
   severity: 'critical',
   message: 'Database connection is down or unhealthy',
   config: {
    type: 'slack',
    enabled: true,
    recipients: ['#alerts'],
   },
  },
  {
   name: 'High Pending Approvals',
   condition: async () => {
    const count = await getPendingApprovalsCount();
    return count > 50;
   },
   severity: 'medium',
   message: 'More than 50 entities pending approval',
   config: {
    type: 'email',
    enabled: true,
    recipients: ['admin@rareminds.com'],
   },
  },
  {
   name: 'Unusual Admin Activity',
   condition: async () => {
    const activityCount = await getAdminActivityCount();
```

```
    return activityCount > 1000; // More than 1000 actions in last hour
   },
   severity: 'high',
   message: 'Unusual admin activity detected (>1000 actions/hour)',
   config: {
    type: 'email',
    enabled: true,
    recipients: ['security@rareminds.com'],
   },
  },
];

// Alert checker (runs periodically)
export async function checkAlerts() {
 for (const rule of alertRules) {
  if (!rule.config.enabled) continue;

  try {
   const shouldAlert = await rule.condition();

   if (shouldAlert) {
    await sendAlert(rule);
   }
  } catch (error) {
   logger.error('Alert check failed', {
    rule: rule.name,
    error: error.message,
   });
  }
 }
}

async function sendAlert(rule: AlertRule) {
 logger.warn('Alert triggered', {
  rule: rule.name,
  severity: rule.severity,
 });

 // Send via configured channels
```

```
  switch (rule.config.type) {
    case 'email':
      await sendEmailAlert(rule);
      break;
    case 'slack':
      await sendSlackAlert(rule);
      break;
    case 'webhook':
      await sendWebhookAlert(rule);
      break;
  }
}
```

15.10 Dashboard Metrics

Key Metrics to Display:

typescript

```
// Types for dashboard metrics
interface DashboardMetrics {
  entities: {
    total: number;
    byType: Record<string, number>;
    byStatus: Record<string, number>;
    pendingApprovals: number;
  };
  users: {
    total: number;
    byRole: Record<string, number>;
    active: number;
    suspended: number;
  };
  activity: {
    todayActions: number;
    weekActions: number;
    monthActions: number;
  };
  performance: {
    avgResponseTime: number;
    errorRate: number;
```

```
  uptime: number;
 };
}

// Fetch dashboard metrics
export async function getDashboardMetrics(): Promise<DashboardMetrics> {
 const [entities, users, activity, performance] = await Promise.all([
  getEntityMetrics(),
  getUserMetrics(),
  getActivityMetrics(),
  getPerformanceMetrics(),
 ]);

 return {
  entities,
  users,
  activity,
  performance,
 };
}
```

---

16. API Reference Quick Guide

Base URLs

```
Production:  https://admin-api.rareminds.com
Staging:     https://admin-api-staging.rareminds.com
Development: http://localhost:8787
```

Authentication

All API requests require JWT token in Authorization header:

bash

```
Authorization: Bearer <jwt_token>
```

Response Format

Success Response:

json

```
{
 "success": true,
 "data": { /* response data */ },
 "message": "Operation successful"
}
```

Error Response:

json

```
{
 "success": false,
 "error": {
  "code": "ERROR_CODE",
  "message": "Human readable error message",
  "details": { /* additional error details */ }
 }
}
```

Common Status Codes

```
200 - OK
201 - Created
400 - Bad Request (validation error)
401 - Unauthorized (invalid/missing token)
403 - Forbidden (insufficient permissions)
404 - Not Found
429 - Too Many Requests (rate limit)
500 - Internal Server Error
503 - Service Unavailable
```

Pagination

For list endpoints that support pagination:

Query Parameters:

```
?page=1&limit=20&sort=created_at&order=desc
```

Response:

```
json
{
 "success": true,
 "data": {
  "items": [...],
  "pagination": {
   "page": 1,
   "limit": 20,
   "total": 150,
   "totalPages": 8,
   "hasMore": true
  }
 }
}
```

Filtering

Query Parameters:
```
?status=active&city=Mumbai&dateFrom=2024-01-01&dateTo=2024-12-31
```

Key Endpoints Summary

Method	Endpoint	Description
Authentication		
POST	`/api/auth/login`	Admin login
POST	`/api/auth/logout`	Logout
POST	`/api/auth/refresh`	Refresh token
GET	`/api/auth/me`	Get current user
Dashboard		
GET	`/api/admin/dashboard`	Dashboard stats
GET	`/api/admin/stats`	Detailed analytics
Schools		
GET	`/api/admin/schools`	List schools
POST	`/api/admin/schools`	Create school

Method	Endpoint	Description
GET	`/api/admin/schools/:id`	Get school details
PUT	`/api/admin/schools/:id`	Update school
DELETE	`/api/admin/schools/:id`	Delete school
POST	`/api/admin/schools/:id/approve`	Approve school
POST	`/api/admin/schools/:id/reject`	Reject school
POST	`/api/admin/schools/:id/suspend`	Suspend school
Colleges		
GET	`/api/admin/colleges`	List colleges
POST	`/api/admin/colleges`	Create college
GET	`/api/admin/colleges/:id`	Get college details
PUT	`/api/admin/colleges/:id`	Update college
DELETE	`/api/admin/colleges/:id`	Delete college
POST	`/api/admin/colleges/:id/approve`	Approve college
Universities		
GET	`/api/admin/universities`	List universities
POST	`/api/admin/universities`	Create university
GET	`/api/admin/universities/:id`	Get university details
PUT	`/api/admin/universities/:id`	Update university
DELETE	`/api/admin/universities/:id`	Delete university
POST	`/api/admin/universities/:id/approve`	Approve university
Companies		
GET	`/api/admin/companies`	List companies
POST	`/api/admin/companies`	Create company
GET	`/api/admin/companies/:id`	Get company details
PUT	`/api/admin/companies/:id`	Update company
DELETE	`/api/admin/companies/:id`	Delete company
POST	`/api/admin/companies/:id/approve`	Approve company
Users		
GET	`/api/admin/users`	List all users
GET	`/api/admin/users/:id`	Get user details
PUT	`/api/admin/users/:id`	Update user
DELETE	`/api/admin/users/:id`	Delete user
POST	`/api/admin/users/:id/suspend`	Suspend user
POST	`/api/admin/users/:id/activate`	Activate user
Approvals		
GET	`/api/admin/approvals/pending`	All pending approvals
GET	`/api/admin/approvals/schools`	Pending schools
POST	`/api/admin/approvals/bulk-action`	Bulk approve/reject
Audit Logs		
GET	`/api/admin/audit-logs`	List audit logs

Method	Endpoint	Description
GET	`/api/admin/audit-logs/:id`	Get audit log details
GET	`/api/admin/audit-logs/export`	Export logs as CSV

---

17. Troubleshooting Guide

Common Issues and Solutions

Issue 1: Cannot Login

Symptoms:

- Login button shows loading indefinitely
- "Invalid credentials" error
- "Network error" message

Solutions:

bash

```
# 1. Check if Admin API is running
curl https://admin-api.rareminds.com/health

# 2. Verify credentials
# Ensure the user exists in database with role='platform_admin'

# 3. Check browser console for errors
# Open DevTools (F12) > Console tab

# 4. Verify environment variables
# Check NEXT_PUBLIC_API_URL is correct

# 5. Clear cookies and try again
# Application tab > Cookies > Clear all

# 6. Check network tab
# DevTools > Network tab > Look for failed requests
```

Issue 2: Token Expired Error

Symptoms:

- "Session expired" message
- Redirected to login page frequently

- 401 Unauthorized errors

Solutions:

typescript

```typescript
// Check token expiry time
const token = getCookie('admin_token');
const decoded = jwtDecode(token);
console.log('Token expires at:', new Date(decoded.exp * 1000));

// If token refresh is failing, check:
// 1. Refresh token exists
// 2. Refresh endpoint is working
// 3. JWT_SECRET matches between frontend and backend
```

Issue 3: Pending Approvals Not Showing

Symptoms:

- Dashboard shows 0 pending approvals
- But entities exist with approval_status='pending'

Solutions:

sql

```sql
-- Verify pending entities in database
SELECT
  'school' as type, COUNT(*) as count
FROM schools WHERE approval_status = 'pending'
UNION ALL
SELECT
  'college' as type, COUNT(*) as count
FROM colleges_standalone WHERE approval_status = 'pending'
UNION ALL
SELECT
  'university' as type, COUNT(*) as count
FROM universities WHERE approval_status = 'pending'
UNION ALL
SELECT
  'company' as type, COUNT(*) as count
FROM companies WHERE approval_status = 'pending';
```

```
-- Check API endpoint
curl -H "Authorization: Bearer <token>" \
  https://admin-api.rareminds.com/api/admin/approvals/pending

-- Clear React Query cache
// In browser console
queryClient.invalidateQueries(['approvals']);
```

Issue 4: Real-Time Notifications Not Working

Symptoms:

- No notification bell updates
- Missing toast notifications
- WebSocket connection failures

Solutions:

typescript

```
// Check WebSocket connection
// In browser console
console.log('WebSocket state:', ws.readyState);
// 0 = CONNECTING, 1 = OPEN, 2 = CLOSING, 3 = CLOSED

// Check WebSocket URL
console.log('WS URL:', process.env.NEXT_PUBLIC_WS_URL);

// Manually test WebSocket
const ws = new WebSocket('wss://admin-ws.rareminds.com');
ws.onopen = () => console.log('Connected');
ws.onerror = (e) => console.error('Error:', e);

// Check if WebSocket server is running
// Backend API should have WebSocket endpoint enabled
```

Issue 5: Slow Page Load

Symptoms:

- Dashboard takes >5 seconds to load
- Entity lists load slowly
- Images/assets slow to load

Solutions:

typescript

```typescript
// 1. Check API response times
console.time('dashboard-api');
await fetch('/api/admin/dashboard');
console.timeEnd('dashboard-api');

// 2. Enable React Query devtools to see query states
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';

// 3. Check if data is being refetched unnecessarily
// Add staleTime to queries
useQuery({
  queryKey: ['dashboard'],
  queryFn: getDashboardData,
  staleTime: 5 * 60 * 1000, // 5 minutes
});

// 4. Implement pagination/infinite scroll
// Instead of loading all data at once

// 5. Use React.lazy for code splitting
const Analytics = React.lazy(() => import('./analytics/page'));

// 6. Optimize images
// Use Next.js Image component
import Image from 'next/image';
```

Issue 6: Build Failures

Symptoms:

- `pnpm build` fails
- TypeScript errors
- Missing dependencies

Solutions:

bash

```bash
# 1. Clear node_modules and reinstall
rm -rf node_modules pnpm-lock.yaml
pnpm install
```

```
# 2. Check for TypeScript errors
pnpm type-check

# 3. Fix linting errors
pnpm lint --fix

# 4. Check Next.js config
# Ensure next.config.js is valid

# 5. Check environment variables
# Ensure all NEXT_PUBLIC_* vars are set

# 6. Check for circular dependencies
# Use madge tool
npx madge --circular src/

# 7. Update dependencies
pnpm update
```

Issue 7: Deployment Failed on Cloudflare

Symptoms:

- Wrangler deploy command fails
- Build succeeds locally but fails on Cloudflare
- "Invalid configuration" error

Solutions:

bash

```
# 1. Check wrangler.toml syntax
wrangler pages project list

# 2. Verify Cloudflare credentials
wrangler whoami

# 3. Check build output directory
# Ensure .next directory exists after build

# 4. Check environment variables in Cloudflare
# Pages > Settings > Environment variables
```

```
# 5. Check build logs in Cloudflare Dashboard
# Pages > Deployments > View logs

# 6. Try manual deployment
pnpm build
wrangler pages deploy .next --project-name=rareminds-admin

# 7. Check for size limits
# Cloudflare Pages has limits on file sizes
# Optimize assets if needed
```

Issue 8: Approve/Reject Actions Not Working

Symptoms:

- Click approve button, nothing happens
- Error toast appears
- Entity status doesn't change

Solutions:

typescript

```
// 1. Check API response
// Open DevTools > Network tab
// Look for /approve or /reject request
// Check response body for errors

// 2. Verify permissions
// Check if admin user has required permissions
console.log('User permissions:', user.permissions);

// 3. Check if entity is already approved/rejected
// Database state might be out of sync with UI

// 4. Check audit logs
// See if action was logged
SELECT * FROM audit_logs
WHERE resource_id = 'entity-id'
ORDER BY created_at DESC
LIMIT 5;

// 5. Invalidate cache after action
```

```
queryClient.invalidateQueries(['schools']);
queryClient.invalidateQueries(['approvals']);
```

---

18. Best Practices

18.1 Code Organization

typescript

```
// ☑ Good: Clear separation of concerns
src/
  components/    // Reusable UI components
  pages/         // Page components
  lib/           // Business logic, utilities
  types/         // TypeScript types
  styles/        // Global styles

// ☑ Good: Colocate related files
schools/
  page.tsx
  components/
    school-table.tsx
    school-form.tsx
  hooks/
    use-schools.ts
  utils/
    validate-school.ts

// ✕ Bad: Everything in one place
src/
  components/
    everything-here.tsx
```

18.2 State Management

typescript

```
// ☑ Good: Use React Query for server state
const { data: schools } = useQuery({
  queryKey: ['schools'],
  queryFn: getSchools,
```

```
});

// ☑ Good: Use Zustand for global UI state
const useSidebarStore = create((set) => ({
  isOpen: true,
  toggle: () => set((state) => ({ isOpen: !state.isOpen })),
}));

// ✗ Bad: Mixing server state with local state
const [schools, setSchools] = useState([]);
useEffect(() => {
  fetchSchools().then(setSchools);
}, []);
```

18.3 Error Handling

typescript

```
// ☑ Good: Comprehensive error handling
try {
  const result = await approveSchool(schoolId);
  toast.success('School approved successfully');
  queryClient.invalidateQueries(['schools']);
} catch (error) {
  if (error instanceof APIError) {
    toast.error(error.message);
  } else {
    toast.error('An unexpected error occurred');
    logger.error('Failed to approve school', {
      schoolId,
      error: error.message,
    });
  }
}

// ✗ Bad: Silent failures
try {
  await approveSchool(schoolId);
} catch (error) {
  // Nothing
}
```

18.4 Performance

typescript

```typescript
// ☑ Good: Memoize expensive computations
const sortedSchools = useMemo(() => {
  return schools.sort((a, b) => a.name.localeCompare(b.name));
}, [schools]);

// ☑ Good: Debounce search inputs
const debouncedSearch = useDebounce(searchQuery, 500);

useEffect(() => {
  searchSchools(debouncedSearch);
}, [debouncedSearch]);

// ☑ Good: Use React.lazy for code splitting
const Analytics = React.lazy(() => import('./analytics'));

// ✕ Bad: Recreating functions on every render
function Component() {
  const handleClick = () => {
    // This creates a new function on every render
  };
  return <Button onClick={handleClick} />;
}
```

18.5 Security

typescript

```typescript
// ☑ Good: Validate all inputs
const schoolSchema = z.object({
  name: z.string().min(2).max(255),
  email: z.string().email(),
});

const validData = schoolSchema.parse(formData);

// ☑ Good: Sanitize HTML if needed
import DOMPurify from 'dompurify';
const clean = DOMPurify.sanitize(html);
```

```
// ❌ Bad: Trust user input
<div dangerouslySetInnerHTML={{ __html: userInput }} />
```

18.6 Testing

typescript

```
// ☑ Good: Test critical flows
describe('School Approval', () => {
 it('should approve school and send notification', async () => {
  const school = createMockSchool();
  await approveSchool(school.id);

  expect(school.approval_status).toBe('approved');
  expect(notificationSent).toBe(true);
 });
});

// ☑ Good: Test error cases
it('should handle approval failure gracefully', async () => {
 mockAPI.approveSchool.mockRejectedValue(new Error('DB error'));

 await expect(approveSchool('id')).rejects.toThrow();
});
```

---

19. Maintenance & Support

19.1 Regular Maintenance Tasks

Daily:

- Monitor error logs in Sentry
- Check system health dashboard
- Review pending approvals count
- Check for any failed deployments

Weekly:

- Review audit logs for unusual activity
- Check database performance
- Review and respond to user feedback
- Update documentation if needed

Monthly:

- Update dependencies (`pnpm update`)
- Review and optimize slow queries
- Check storage usage
- Review and archive old audit logs
- Security audit
- Performance review

Quarterly:

- Major dependency updates
- Security vulnerability assessment
- Backup restoration testing
- Disaster recovery drill
- Review and update permissions
- Code quality review
- Performance benchmarking
- User access audit
- Review and optimize cloud costs

Annually:

- Comprehensive security audit
- Architecture review
- Database cleanup (archive old data)
- SSL certificate renewal (if not auto-renewed)
- Review and update documentation
- Team training on new features
- Infrastructure cost optimization

19.2 Backup Strategy

Database Backups:

sql

```sql
-- Supabase provides automatic daily backups
-- Additional manual backup script

-- Export critical tables
COPY (SELECT * FROM users) TO '/backups/users_backup.csv' CSV HEADER;
COPY (SELECT * FROM schools) TO '/backups/schools_backup.csv' CSV HEADER;
COPY (SELECT * FROM colleges_standalone) TO '/backups/colleges_backup.csv' CSV HEADER;
COPY (SELECT * FROM universities) TO '/backups/universities_backup.csv' CSV HEADER;
COPY (SELECT * FROM companies) TO '/backups/companies_backup.csv' CSV HEADER;
COPY (SELECT * FROM audit_logs) TO '/backups/audit_logs_backup.csv' CSV HEADER;
```

```
-- Or use Supabase CLI
supabase db dump -f backup_$(date +%Y%m%d).sql
```

Backup Schedule:

- **Automatic Daily:** Handled by Supabase (retained for 7 days on free tier, 30+ days on paid)
- **Weekly Manual:** Full database export to secure storage
- **Monthly Archive:** Long-term storage of audit logs and critical data
- **Before Major Changes:** Manual backup before deploying significant updates

Backup Locations:

- Primary: Supabase automatic backups
- Secondary: AWS S3 or Google Cloud Storage
- Tertiary: Local encrypted storage (for disaster recovery)

Backup Testing:

bash

```
# Test backup restoration monthly
# 1. Create test database
supabase db create test-restore

# 2. Restore from backup
supabase db restore backup_20240101.sql --db test-restore

# 3. Verify data integrity
supabase db query "SELECT COUNT(*) FROM users" --db test-restore

# 4. Drop test database
supabase db drop test-restore
```

19.3 Disaster Recovery Plan

Recovery Time Objective (RTO): 2 hours
Recovery Point Objective (RPO): 24 hours (daily backups)

Disaster Scenarios & Recovery Steps:

Scenario 1: Database Corruption

bash

```
# Step 1: Identify the issue
# Check database health
```

```
supabase db health

# Step 2: Stop all write operations
# Set maintenance mode in Admin App

# Step 3: Identify last known good backup
supabase db list-backups

# Step 4: Restore from backup
supabase db restore backup_20240101.sql

# Step 5: Verify data integrity
# Run integrity checks on critical tables

# Step 6: Resume operations
# Disable maintenance mode

# Step 7: Post-mortem
# Document what happened and how to prevent it
```

Scenario 2: Cloudflare Pages Outage

bash

```
# Cloudflare has 100% uptime SLA, but in case of issues:

# Step 1: Check Cloudflare Status
# Visit: status.cloudflare.com

# Step 2: If prolonged outage, deploy to alternative
# Deploy to Vercel as backup
vercel --prod

# Step 3: Update DNS if needed
# Point admin.rareminds.com to Vercel temporarily

# Step 4: Monitor recovery
# Switch back to Cloudflare once resolved
```

Scenario 3: Admin API Failure

bash

```
# Step 1: Check health endpoint
curl https://admin-api.rareminds.com/health

# Step 2: Check Cloudflare Workers status
wrangler tail

# Step 3: If worker is down, redeploy
cd admin-api
wrangler deploy --env production

# Step 4: If database connection issue
# Check Supabase connection strings
# Verify service keys are valid

# Step 5: If still failing, rollback to previous version
wrangler rollback
```

Scenario 4: Data Breach or Security Incident

bash

```
# Step 1: Immediately revoke all JWT tokens
# Invalidate refresh tokens in database
UPDATE users SET last_password_change = NOW()
WHERE role = 'platform_admin';

# Step 2: Force password reset for all admins
# Send password reset emails

# Step 3: Review audit logs
SELECT * FROM audit_logs
WHERE created_at > (NOW() - INTERVAL '7 days')
ORDER BY created_at DESC;

# Step 4: Identify compromised data
# Check what was accessed/modified

# Step 5: Notify affected parties
# Send notifications to entity admins if their data was accessed

# Step 6: Patch security vulnerability
```

```
# Deploy fix immediately

# Step 7: Incident report
# Document breach, impact, and remediation
```

19.4 Version Control & Release Management

Branching Strategy:
```
main (production)
  ├── staging (pre-production)
  ├── develop (development)
  └── feature/* (feature branches)
      ├── feature/entity-approval-v2
      ├── feature/analytics-dashboard
      └── bugfix/login-timeout
```

Release Process:

```bash
# 1. Create feature branch
git checkout -b feature/new-feature

# 2. Develop and commit changes
git add .
git commit -m "feat: add new feature"

# 3. Push to remote
git push origin feature/new-feature

# 4. Create Pull Request to develop
# Review and merge after CI passes

# 5. When ready for staging
git checkout staging
git merge develop
git push origin staging

# 6. Test on staging environment
```

 # Run manual and automated tests

 # 7. When ready for production
git checkout main
git merge staging
git tag v1.2.0
git push origin main --tags

 # 8. Monitor deployment
 # Check logs and metrics

 # 9. If issues, rollback
git revert HEAD
git push origin main
```

**Semantic Versioning:**
```
v[MAJOR].[MINOR].[PATCH]

Examples:
v1.0.0 - Initial release
v1.1.0 - New features (backward compatible)
v1.1.1 - Bug fixes
v2.0.0 - Breaking changes

Commit message prefixes:
feat: New feature
fix: Bug fix
docs: Documentation
style: Code style (formatting)
refactor: Code refactoring
test: Adding tests
chore: Maintenance
```

**Release Checklist:**
```
☐ All tests passing

☐ Code reviewed and approved

☐ Documentation updated

☐ CHANGELOG.md updated

☐ Version bumped in package.json

☐ Database migrations (if any) tested

☐ Staging deployment successful

☐ Smoke tests passed on staging

☐ Rollback plan prepared

☐ Stakeholders notified

☐ Deploy to production

☐ Monitor for 30 minutes post-deployment

☐ Send release notes to team

19.5 Support Procedures

Support Channels:

1. **Internal Team Support:**
    - Slack channel: #admin-app-support
    - Email: admin-support@rareminds.com
    - Response time: 1 hour during business hours
2. **Technical Support:**
    - GitHub Issues for bugs
    - Email: tech@rareminds.com
    - Response time: 4 hours
3. **Emergency Support:**
    - Phone: +91-XXXX-XXXXXX (on-call engineer)
    - Available 24/7 for critical issues
    - Response time: 15 minutes

Support Tiers:

Tier 1 - Critical (P0):

- Admin App completely down
- Security breach
- Data loss
- Response: Immediate
- Resolution: Within 2 hours

Tier 2 - High (P1):

- Major feature not working
- Performance severely degraded
- Response: Within 1 hour

- Resolution: Within 4 hours

Tier 3 - Medium (P2):

- Minor feature issues
- Cosmetic bugs
- Response: Within 4 hours
- Resolution: Within 2 days

Tier 4 - Low (P3):

- Feature requests
- Documentation issues
- Response: Within 1 day
- Resolution: Next release cycle

Support Ticket Format:

markdown

```
### Issue Description
[Describe the issue clearly]

### Steps to Reproduce
1. Go to...
2. Click on...
3. See error...

### Expected Behavior
[What should happen]

### Actual Behavior
[What actually happened]

### Environment
- Browser: Chrome 120
- OS: Windows 11
- Admin App Version: v1.2.0
- User Role: platform_admin

### Screenshots/Logs
[Attach if available]

### Priority
```

[P0/P1/P2/P3]
```

### 19.6 Monitoring & Alerting Checklist

**What to Monitor:**
```
Application Metrics:
☐ Error rate (< 1%)
☐ Response time (< 500ms p95)
☐ Request rate
☐ CPU usage (< 80%)
☐ Memory usage (< 80%)

Business Metrics:
☐ Active admin users
☐ Pending approvals count
☐ Daily approval rate
☐ Entity creation rate
☐ Failed login attempts

Infrastructure Metrics:
☐ Database connection pool
☐ Database query performance
☐ CDN cache hit rate
☐ WebSocket connections
☐ Storage usage

Security Metrics:
☐ Failed authentication attempts
☐ Unusual activity patterns
☐ Permission violations
☐ Suspicious IP addresses
```

**Alert Thresholds:**

```typescript
const ALERT_THRESHOLDS = {
```
```

```
  errorRate: 0.05,         // 5%
  responseTime: 2000,      // 2 seconds (p95)
  failedLogins: 10,        // per hour per user
  pendingApprovals: 100,   // entities
  databaseConnections: 90,  // % of pool
  storageUsage: 85,        // % of quota
};
```

19.7 Documentation Updates

Documentation Maintenance:

bash

```
# Keep documentation in sync with code
docs/
  ├── ARCHITECTURE.md       # Updated when architecture changes
  ├── API.md                # Updated when API changes
  ├── DEPLOYMENT.md         # Updated when deployment process changes
  ├── TROUBLESHOOTING.md    # Updated when new issues discovered
  └── CHANGELOG.md          # Updated with every release

# Documentation review schedule
- Weekly: Review pending updates
- Monthly: Comprehensive review
- Quarterly: Major documentation refresh
```

Documentation Standards:

markdown

```
# Use clear headings
## Main sections with ##
### Subsections with ###

# Include code examples
```typescript
// Always add comments to code examples
const example = 'like this';
```

Add visual aids where helpful
[diagram or screenshot]
```

Keep it up-to-date

Last updated: 2024-01-15

Link to related docs

See also: [Related Documentation](./related.md)

---

20. Future Enhancements

20.1 Planned Features

Short-term (Next 3 months):

1. **Advanced Analytics Dashboard**
   - Custom date range selection
   - Drill-down capabilities
   - Export analytics reports
   - Comparison views (YoY, MoM)
   - Predictive analytics
2. **Bulk Operations Enhancement**
   - CSV import for bulk entity creation
   - Bulk edit capabilities
   - Scheduled bulk actions
   - Bulk validation and preview
3. **Enhanced Notifications**
   - Email notifications
   - SMS alerts for critical events
   - Notification preferences
   - Digest emails (daily/weekly summary)
4. **Audit Log Improvements**
   - Advanced filtering
   - Visual diff viewer
   - Audit log retention policies
   - Automated compliance reports
5. **Role Management UI**
   - Custom role creation
   - Permission assignment interface
   - Role hierarchy visualization
   - Permission testing tool

Mid-term (3-6 months):

1. **Multi-Admin Support**
   - Create sub-admin accounts
   - Delegate specific permissions
   - Admin activity tracking
   - Admin performance metrics

2. **Automated Compliance**
    - GDPR compliance tools
    - Data retention policies
    - Automated data exports
    - Right to be forgotten implementation
3. **Advanced Search**
    - Full-text search across all entities
    - Saved search queries
    - Search history
    - Fuzzy matching
4. **Workflow Automation**
    - Auto-approval based on criteria
    - Scheduled tasks
    - Webhook integrations
    - Custom workflow builder
5. **Mobile Admin App**
    - React Native mobile app
    - Push notifications
    - Quick approval actions
    - Mobile-optimized dashboard

Long-term (6-12 months):

1. **AI-Powered Features**
    - Anomaly detection in audit logs
    - Predictive entity approval (risk scoring)
    - Smart recommendations
    - Automated report generation
2. **Advanced Security**
    - Two-factor authentication
    - Biometric authentication
    - IP whitelisting
    - Session recording
3. **API for Third-Party Integrations**
    - Public API documentation
    - API key management
    - Rate limiting per API key
    - Webhook system
4. **Multi-tenancy**
    - Multiple RM Admin instances
    - Region-specific deployments
    - Data residency compliance
    - Tenant isolation
5. **Advanced Reporting**
    - Custom report builder
    - Scheduled reports
    - Report templates
    - Interactive dashboards

20.2 Technical Improvements

Performance Optimizations:

- Implement Redis caching layer
- Database query optimization
- CDN optimization
- Lazy loading for large datasets
- Image optimization and compression

Code Quality:

- Increase test coverage to 80%
- Implement E2E test suite
- Code quality gates in CI/CD
- Automated code reviews
- Performance budgets

Developer Experience:

- Storybook for component library
- Better error messages
- Development documentation
- Local development improvements
- Debugging tools

---

21. Glossary

Entity: A registered organization (School, College, University, or Company) on the Rareminds platform.

Entity Type: The category of an entity (school, college_standalone, university, company).

Entity Admin: The primary administrator account for an entity (e.g., School Admin, College Admin).

Platform Admin (RM Admin): The top-level administrator with access to the Admin App and full platform control.

Approval Status: The current state of an entity's registration request (pending, approved, rejected).

Account Status: The operational state of an entity or user account (active, inactive, suspended, pending).

Audit Log: A record of all actions performed on the platform, including who did what, when, and from where.

JWT (JSON Web Token): A secure token used for authentication and authorization.

RBAC (Role-Based Access Control): A security model where permissions are assigned to roles, and users are assigned roles.

Permission: A specific action that a user is allowed to perform (e.g., school:approve, user:suspend).

Supabase: The PostgreSQL database service used for data storage and authentication.

Cloudflare Pages: The hosting platform for the Admin App frontend.

Cloudflare Workers: The serverless compute platform for the Admin API backend.

Hono.js: The lightweight web framework used for the Admin API.

shadcn/ui: The UI component library used in the Admin App.

React Query (TanStack Query): The data fetching and caching library used in the Admin App.

Zustand: The state management library for global UI state.

Zod: The schema validation library for TypeScript.

WebSocket: A protocol for real-time bidirectional communication between client and server.

Infinite Scroll: A pagination technique where more data loads automatically as the user scrolls.

Server-Side Sorting: Sorting data on the backend API rather than in the browser.

Idempotent: An operation that produces the same result no matter how many times it's executed.

---

22. Support & Contact Information

Development Team

Lead Developer:

- Name: [Your Name]
- Email: lead.dev@rareminds.com
- Availability: Monday-Friday, 9 AM - 6 PM IST

Backend Team:

- Email: backend@rareminds.com
- Slack: #backend-team

Frontend Team:

- Email: frontend@rareminds.com
- Slack: #frontend-team

DevOps Team:

- Email: devops@rareminds.com
- Slack: #devops-team
- On-call: +91-XXXX-XXXXXX

External Services

Supabase Support:

- Dashboard: https://app.supabase.com
- Docs: https://supabase.com/docs
- Support: support@supabase.io

Cloudflare Support:

- Dashboard: https://dash.cloudflare.com
- Docs: https://developers.cloudflare.com
- Support: https://support.cloudflare.com

Sentry Support:

- Dashboard: https://sentry.io
- Docs: https://docs.sentry.io
- Support: support@sentry.io

Emergency Contacts

Critical Issues (24/7):

- On-call Engineer: +91-XXXX-XXXXXX
- Backup: +91-YYYY-YYYYYY

Security Issues:

- Email: security@rareminds.com
- Response time: Immediate

Data Breach:

- Email: security@rareminds.com
- Phone: +91-XXXX-XXXXXX
- Escalation: CTO

---

23. Appendix

23.1 Common Commands Reference

bash

```bash
# Development
pnpm dev                # Start development server
pnpm build              # Build for production
pnpm start              # Start production server
pnpm lint               # Lint code
pnpm lint:fix           # Fix linting issues
pnpm type-check         # TypeScript type checking
pnpm format             # Format code with Prettier

# Testing
pnpm test               # Run unit tests
pnpm test:watch         # Run tests in watch mode
pnpm test:coverage      # Generate coverage report
pnpm test:e2e           # Run E2E tests

# Database
supabase db pull        # Pull remote schema
supabase db push        # Push local schema
supabase db reset       # Reset database
supabase db seed        # Run seed scripts
supabase db dump        # Export database

# Deployment
wrangler login          # Login to Cloudflare
wrangler pages deploy   # Deploy to Cloudflare Pages
wrangler pages list     # List deployments
wrangler tail           # View logs

# Git
git status              # Check status
git add .               # Stage changes
git commit -m "message"  # Commit changes
git push                # Push to remote
git pull                # Pull from remote
```

```
git checkout -b feature   # Create feature branch
```

23.2 Environment Variables Reference

bash

```
# Required
NEXT_PUBLIC_API_URL              # Admin API URL
NEXT_PUBLIC_SUPABASE_URL         # Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY    # Supabase anonymous key

# Optional
NEXT_PUBLIC_WS_URL               # WebSocket URL
NEXT_PUBLIC_SENTRY_DSN           # Sentry error tracking
NEXT_PUBLIC_ENVIRONMENT          # Environment name
NEXT_PUBLIC_APP_VERSION          # App version
NEXT_PUBLIC_ENABLE_ANALYTICS     # Enable/disable analytics
NEXT_PUBLIC_ENABLE_NOTIFICATIONS # Enable/disable notifications
```

23.3 Key Files Reference

```
.env.local              # Local environment variables
.env.production         # Production environment variables
next.config.js          # Next.js configuration
tailwind.config.ts      # Tailwind CSS configuration
tsconfig.json           # TypeScript configuration
wrangler.toml           # Cloudflare configuration
package.json            # Dependencies and scripts
.eslintrc.json          # ESLint configuration
.prettierrc             # Prettier configuration
```

23.4 Useful Links

Project Resources:

- GitHub Repository: https://github.com/rareminds/admin-app
- Admin App (Production): https://admin.rareminds.com
- Admin App (Staging): https://admin-staging.rareminds.com
- Admin API (Production): https://admin-api.rareminds.com
- Admin API (Staging): https://admin-api-staging.rareminds.com

Documentation:

- Main Architecture: ./ARCHITECTURE.md
- API Documentation: ./API.md
- Deployment Guide: ./DEPLOYMENT.md
- User Flows: ./USER_FLOWS.md

External Resources:

- Next.js Docs: https://nextjs.org/docs
- React Docs: https://react.dev
- TailwindCSS Docs: https://tailwindcss.com/docs
- shadcn/ui Docs: https://ui.shadcn.com
- Supabase Docs: https://supabase.com/docs
- Cloudflare Docs: https://developers.cloudflare.com
- Hono.js Docs: https://hono.dev

Document Information

Document Title: Rareminds Admin App - Complete Documentation
Version: 1.0
Last Updated: October 31, 2025
Author: Rareminds Development Team
Document Type: Technical Documentation
Confidentiality: Internal Use Only

Change Log

Version	Date	Author	Changes
1.0	2025-10-31	Dev Team	Initial documentation

End of Document

This comprehensive documentation covers all aspects of the Rareminds Admin App, from architecture and setup to deployment and maintenance. It should serve as the definitive reference for developers, administrators, and stakeholders working with the Admin App.